Desafio Final — Trilha Engenharia de Dados Júnior

Autora: Nayane Araujo
Repositório Oficial no GitHub: https://github.com/Nayanearaujo/desafio-engenharia-dados-jr
Pull Requests Mergeados: PR #1 | PR #2 | PR #3
LinkedIn: linkedin.com/in/nayane-oaraujo | **Portfólio:** nayanearaujo.github.io

Loja Digital: Pipeline de Engenharia de Dados em Arquitetura Medalha

Autora: Nayane Araujo · **Repositório GitHub:** github.com/Nayanearaujo/desafio-engenharia-dados-jr · **LinkedIn:** linkedin.com/in/nayane-oaraujo · **Portfólio:** nayanearaujo.github.io

1. Sobre o Projeto e Tecnologias Utilizadas

Neste projeto, construí e evoluí um pipeline de dados ponta a ponta e um painel analítico para a **Loja Digital**, um e-commerce de produtos digitais voltado à área de tecnologia (cursos, e-books e templates).

A versão inicial atendia aos requisitos do Desafio Final da Trilha de Engenharia de Dados Júnior: aplicação em Python e SQLite com menu interativo no terminal (opções 1 a 14), modelagem relacional, consultas SQL (`JOIN` , `GROUP BY` , `HAVING` , `Subquery` , `CTE` , `Window Functions` e `View`) e 14 testes unitários.

Para evoluir a solução mantendo a execução local simples e 100% de compatibilidade com os testes originais, implementei uma **Arquitetura Medalha (Bronze, Silver e Gold)** com separação explícita entre geração sintética (`--pipeline`) e reprocessamento ETL (`--reprocessar`), quarentena auditável de registros inválidos (`silver_rejeitados`), histórico persistido de execuções (`pipeline_execucoes`) e camada analítica Gold reconciliada.

Camada / Componente	Tecnologias e Recursos Aplicados
Linguagem e Tipagem	Python 3.11+, Type Hints, `argparse` , `logging` , configuração centralizada (`src/config.py`)
Banco de Dados e SQL	SQLite 3 (`PRAGMA foreign_keys = ON`), DDL relacional, `VIEWs` , `CTEs` , `Window Functions` (`LAG` , `RANK` , `NTILE` , `SUM/AVG OVER`)
Arquitetura de Dados	Medalha (`Bronze -> Silver -> Gold`), Quarentena (`silver_rejeitados`) e Auditoria de Execuções (`pipeline_execucoes`)
Análise e Visualização	`pandas` (séries temporais e Carta de Controle I-MR dessazonalizada), `matplotlib` (Dashboard Executivo PNG)
Qualidade e CI/CD	`pytest` (30 testes automatizados), `pytest-cov` (90%+ de cobertura em `src/`), `ruff` (lint) e **GitHub Actions**

---

2. Demonstração Visual e Evidência de Execução

O painel abaixo é gerado automaticamente em `assets/dashboard_executivo.png` ao executar `python src/main.py --pipeline` (ou na opção `13` do menu interativo):

Ver saída real da execução do pipeline e da suíte de testes no terminal

```
$ python src/main.py --pipeline
Inicializando banco de dados...
Banco de dados pronto!

2026-10-23 16:00:01 [INFO] pipeline.gerar_dados: Arquivo raw gerado: clientes.csv (205 linhas)
2026-10-23 16:00:01 [INFO] pipeline.gerar_dados: Arquivo raw gerado: produtos.csv (34 linhas)
2026-10-23 16:00:01 [INFO] pipeline.gerar_dados: Arquivo raw gerado: pedidos.csv (5075 linhas)
2026-10-23 16:00:01 [INFO] pipeline.gerar_dados: Arquivo raw gerado: itens_pedido.csv (6880 linhas)
2026-10-23 16:00:01 [INFO] pipeline.bronze: Camada Bronze carregada: bronze_clientes (205 linhas)
2026-10-23 16:00:01 [INFO] pipeline.bronze: Camada Bronze carregada: bronze_produtos (34 linhas)
2026-10-23 16:00:01 [INFO] pipeline.bronze: Camada Bronze carregada: bronze_pedidos (5075 linhas)
2026-10-23 16:00:01 [INFO] pipeline.bronze: Camada Bronze carregada: bronze_itens_pedido (6880 linhas)
2026-10-23 16:00:01 [INFO] pipeline.silver: bronze_clientes: 205 linhas -> silver clientes: 200 validas, 5
rejeitadas
2026-10-23 16:00:01 [INFO] pipeline.silver: bronze_produtos: 34 linhas -> silver produtos: 30 validas, 4
rejeitadas
2026-10-23 16:00:01 [INFO] pipeline.silver: bronze_pedidos: 5075 linhas -> silver pedidos: 5000 validas, 75
rejeitadas
2026-10-23 16:00:01 [INFO] pipeline.silver: bronze_itens_pedido: 6880 linhas -> silver itens_pedido: 6820
validas, 60 rejeitadas
2026-10-23 16:00:01 [INFO] pipeline.qualidade: Relatorio de qualidade gerado em exports/qualidade_rejeitados.csv
(144 rejeicoes totais em 18 motivos)
2026-10-23 16:00:01 [INFO] pipeline.gold: Camada Gold atualizada com sucesso (views gold_* prontas).
2026-10-23 16:00:01 [INFO] pipeline.gold: Exportacao Gold concluida em exports (11 arquivos gerados)
2026-10-23 16:00:01 [INFO] pipeline.executar: Execucao registrada em pipeline_execucoes (run_id=1,
status=sucesso, modo=regerado)

$ pytest -q && ruff check src/ tests/
30 passed in 2.41s
All checks passed!
```

---

3. Como Executar (Quickstart)

1. Clonar o repositório e instalar dependências

```
git clone https://github.com/Nayanearaujo/desafio-engenharia-dados-jr.git
cd desafio-engenharia-dados-jr

python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

(Para instalar também as ferramentas de teste e lint: `pip install -r requirements-dev.txt`. No Windows, ative com `.venv\Scripts\activate`.)

2. Quatro modos de execução disponíveis

```
# Modo 1 (Geracao + ETL completo): Gera CSVs sinteticos em data/raw/ e executa Bronze -> Silver -> Gold +
Dashboard PNG
python src/main.py --pipeline

# Modo 2 (Reprocessamento ETL): Reprocessa os CSVs ja existentes em data/raw/ sem regerar dados sinteticos
python src/main.py --reprocessar

# Modo 3 (Menu interativo): Abre o menu (opcoes 1 a 20). Se o .db nao existir ainda, inicializa com seed.sql e
views Gold
python src/main.py

# Modo 4 (Demonstracao enxuta): Reinicializa o banco com o seed.sql pequeno (7 clientes, 7 produtos) e abre o
menu
python src/main.py --demo
```

Ver as 20 opções do menu interativo no terminal (1 a 14 originais + 15 a 20 Arquitetura Medalha)

```
============================================================
  LOJA DIGITAL — PAINEL DE RELATORIOS E ARQUITETURA MEDALHA
============================================================
  [ 1] Listar todos os produtos
  [ 2] Filtrar produtos por categoria
  [ 3] Listar clientes cadastrados
  [ 4] Pedidos detalhados (View SQL)
  [ 5] Ranking de produtos mais vendidos
  [ 6] Faturamento por categoria
  [ 7] Clientes por volume de compras
  [ 8] Clientes acima de meta (HAVING)
  [ 9] Produtos acima da media (Subquery)
  [10] Desempenho dos produtos (CTE + CASE WHEN)
  [11] KPIs executivos e Curva de Pareto (Window Function)
  [12] Cadastrar novo produto
  [13] Gerar Dashboard Executivo (Matplotlib)
  [14] Automatizar exportacao analitica (CSV)
  [15] Executar pipeline Medalha (Bronze -> Silver -> Gold)
  [16] Relatorio de qualidade e quarentena (Silver)
  [17] Receita mensal, MoM e media movel 3M (Gold)
  [18] Segmentacao RFM de clientes (Gold)
  [19] Taxa de recompra e retencao por coorte (Gold)
  [20] Cancelamentos e Carta de Controle 3-Sigma (Gold)
  [ 0] Sair
------------------------------------------------------------
```

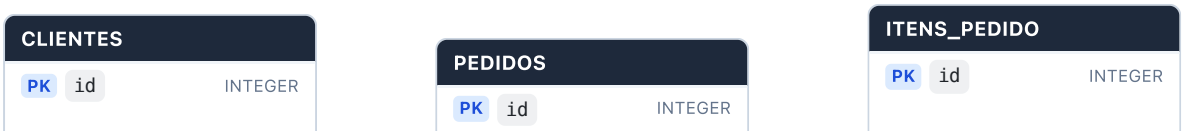
4. Arquitetura Medalha (Bronze, Silver e Gold)

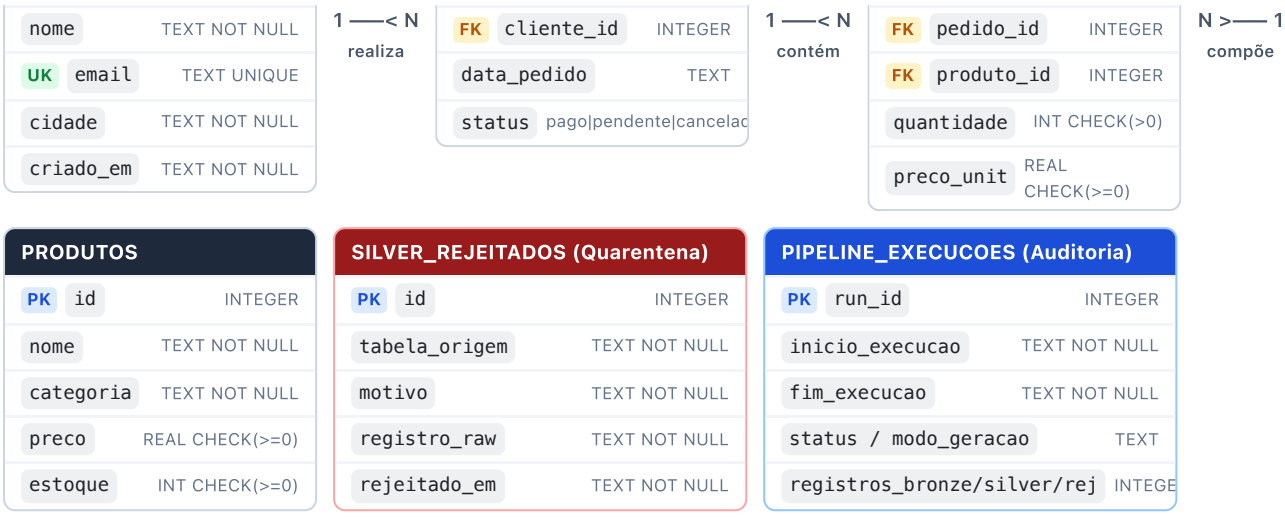
O fluxo de dados segue sentido único (`Bronze -> Silver -> Gold`), com separação explícita entre a geração sintética (`gerar_dados.py`) e a ingestão/transformação ETL (`--reprocessar`). Cada execução é idempotente nas tabelas de destino e grava uma linha de auditoria em `pipeline_execucoes` (`run_id` , timestamps, modo de geração e contagens de registros lidos, válidos e rejeitados).



5. Modelo de Dados Relacional e Auditoria (Camada Silver)

Na camada Silver, o SQLite opera com `PRAGMA foreign_keys = ON` , restrições de domínio (`CHECK` , `NOT NULL` , `UNIQUE`) e tipagem definida. Os registros que violam qualquer regra na transição Bronze para Silver são isolados em `silver_rejeitados` com o registro bruto original (`registro_raw`) e o motivo da rejeição, enquanto `pipeline_execucoes` preserva o histórico de cada processamento.





---

6. Principais Insights, Reconciliação de KPIs e Premissas Estatísticas (Camada Gold)

> **Valores de Referência e Contrato de Status**: os números de referência abaixo são gerados com **semente `42` (`Faker 40.43.0`)**.
> Todos os **KPIs financeiros e de clientes da camada Gold** (`Receita Paga` , `Ticket Médio Pago` , `LTV Médio Pago` , `Receita Mensal MoM` , `Matriz RFM` , `Coorte` , `Taxa de Recompra` , `Pareto Gold` e `Carta de Controle`) filtram estritamente **`status = 'pago'`** , garantindo reconciliação aritmética exata entre todos os painéis. As consultas operacionais legadas da camada Silver (opções 4, 6, 7, 10 e 11 do menu original) mantêm o filtro `status != 'cancelado'` (`pago + pendente = R$ 2.635.945,00`) para preservar o comportamento original dos testes.

Indicador Executivo (Camada Gold)	Fórmula / Base de Cálculo (**`status = 'pago'`**)	Valor Reconciliado
Receita Paga Confirmada	Soma dos itens de `3.830` pedidos pagos (`76,6%` de conversão)	**R$ 2.248.527,00**
Ticket Médio Pago	`R$ 2.248.527,00 ÷ 3.830 pedidos pagos`	**R$ 587,08**
LTV Médio Pago	`R$ 2.248.527,00 ÷ 198 clientes pagantes`	**R$ 11.356,20**
Taxa de Recompra (Pagantes)	`164 clientes recorrentes (≥ 2 compras pagas) ÷ 198 pagantes`	**82,8%**
Curva de Pareto 80/20	`4 de 30 produtos (13,3%)` somam `R$ 1.816.299,00`	**80,8% da receita paga**
Pico Sazonal Mensal (`2026-06`)	Maior faturamento mensal pago (MM3M de `R$ 324.870,33`)	**R$ 366.076,00**
Dispersão de Curto Prazo ($\hat{\sigma}_{IMR}$)	Amplitude móvel dos resíduos (`R$ 2.882,94 ÷ 1,128`)	**R$ 2.555,79**

- **Receita Pendente e Cancelamentos**: além dos `3.830` pedidos pagos (`76,60%`), a base de referência possui **688 pedidos pendentes** (`R$ 387.418,00` / `13,76%`) e **482 pedidos cancelados** (`9,64%` , exibido como `9,6%` nas views). *(2 dos 200 clientes válidos possuem apenas pedidos pendentes/cancelados).*
- **Matriz RFM (100% reconciliada com a Receita Paga)**: a soma dos 4 segmentos fecha exatamente em **R$ 2.248.527,00**: **Campeões** (`63` clientes | **R$ 1.346.807,00** | **59,9%**), **Inativos** (`96` clientes | **R$ 440.472,00** | **19,6%**), **Leais** (`30` clientes | **R$ 376.465,00** | **16,7%**) e **Em risco** (`9` clientes | **R$ 84.783,00** | **3,8%**).

Metodologia e Limitações Estatísticas do Monitoramento de Anomalias (Carta I-MR)

Como a receita diária possui tendência e sazonalidade acentuadas ao longo de 12 meses, uma carta de controle estática baseada no desvio-padrão amostral global (s = R$ 4.589,17) inflaria os limites e mascararia anomalias fora do pico. Para tratar a não-estacionariedade, extraí a tendência sazonal via média móvel centrada de 15 dias ($\hat{T}_t$) e apliquei sobre os resíduos ($e_t = y_t - \hat{T}_t$) o estimador de curto prazo por **Amplitude Móvel (Carta I-MR)**:

$$\hat{\sigma}_{IMR} = \overline{MR} / d_2 = \text{R\$ } 2.882{,}94 / 1{,}128 = \text{R\$ } 2.555{,}79$$

Com a banda dinâmica $\hat{T}_t \pm 3\hat{\sigma}_{IMR}$, o modelo sinaliza exatamente os **2 dias com choques sintéticos injetados na simulação**: `2026-02-28` (`R$ 14.041,00` , $Z = +3{,}16\sigma$) e `2026-07-04` (`R$ 40.227,00` , $Z = +10{,}58\sigma$).

Premissas e limitações conhecidas da abordagem: 1. **Efeito de borda da janela centrada (`window=15, center=True, min_periods=1`)**: nos primeiros e últimos 7 dias da série, a média móvel opera com menos de 15 observações. Além disso, como a janela é centrada (usa 7 dias passados e 7 futuros), ela é adequada para **diagnóstico histórico retrospectivo**, mas em monitoramento online em tempo real deve ser substituída por uma janela causal (retrospectiva) ou alisamento exponencial (EWMA). 2. **Influência de outliers extremos sobre a média móvel e sobre *MR̄***: um pico muito elevado (como `2026-07-04`) puxa levemente para cima a tendência local $\hat{T}_t$ nos 7 dias vizinhos e aumenta o valor médio de $\overline{MR}$. Em séries com múltiplos outliers próximos, uma evolução recomendada é estimar $\hat{T}_t$ por mediana móvel e calcular o limite a partir da mediana da amplitude móvel ($d_4 = 0{,}954$). 3. **Banda dinâmica por dia vs. limites globais de referência**: o dicionário de retorno expõe tanto os limites globais em torno da média geral (`lsc_3sigma` e `lic_3sigma` escalares) para sumário executivo quanto os vetores diários (`df["lsc_3sigma"] = tendencia + 3 * sigma_imr`) utilizados na detecção efetiva de anomalias.

---

7. Validação de Esquema e Quarentena de Dados (Camada Silver)

O módulo `src/pipeline/qualidade.py` aplica regras determinísticas antes da carga na camada Silver: - **Obrigatoriedade e formato canônico de chave primária**: rejeita campos vazios, chaves não numéricas, negativas, repetidas ou em formato não canônico (como `"4.0"`). - **Finitude e positividade numérica**: rejeita valores menores ou iguais a zero e bloqueia explicitamente `NaN` e `inf` via `math.isfinite()` . - **Validação de calendário ISO e janela temporal de negócio**: valida datas reais (`YYYY-MM-DD`) e exige que estejam dentro da janela operacional configurada em `src/config.py` (`2020-01-01` a `2030-12-31`), bloqueando datas fora do intervalo com o motivo `data_fora_janela_negocio` . - **Domínio de status, sintaxe/unicidade de e-mail e integridade referencial (FKs)**: garante que nenhum pedido ou item órfão chegue às tabelas analíticas.

O gerador sintético (`src/pipeline/gerar_dados.py`) injeta **144 registros inválidos em 12.194 linhas brutas (taxa global de 1,18%)**:

Camada / Tabela	Linhas Lidas (Bronze)	Linhas Válidas (Silver)	Linhas em Quarentena (`silver_rejeitados`)	Taxa de Rejeição
`clientes`	205	200	5	2,44%
`produtos`	34	30	4	11,76%
`pedidos`	5.075	5.000	75	1,48%
`itens_pedido`	6.880	6.820	60	0,87%
Total Geral	**12.194**	**12.050**	**144**	**1,18%**

Ver detalhamento dos 144 registros bloqueados por tabela e motivo (18 combinações)

Tabela de Origem	Motivo da Rejeição (`motivo`)	Quantidade	Regra Aplicada em `qualidade.py`
`bronze_pedidos`	`chave_duplicada`	15	Unicidade de chave primária (`id`) no lote

Tabela de Origem	Motivo da Rejeição (`motivo`)	Quantidade	Regra Aplicada em `qualidade.py`
`bronze_pedidos`	`data_invalida:data_pedido`	15	Validação de calendário ISO (`YYYY-MM-DD`)
`bronze_pedidos`	`fk_orfa:cliente_id`	15	Integridade referencial contra `clientes` válidos
`bronze_pedidos`	`nulo_obrigatorio:cliente_id`	15	Preenchimento obrigatório de chave estrangeira
`bronze_pedidos`	`status_fora_do_dominio`	15	Domínio restrito a `('pago', 'pendente', 'cancelado')`
`bronze_itens_pedido`	`chave_duplicada`	12	Unicidade de chave primária (`id`) no lote
`bronze_itens_pedido`	`fk_orfa:pedido_id`	12	Integridade referencial contra `pedidos` válidos
`bronze_itens_pedido`	`fk_orfa:produto_id`	12	Integridade referencial contra `produtos` válidos
`bronze_itens_pedido`	`valor_menor_ou_igual_zero:preco_unit`	12	Regra numérica estrita (`preco_unit > 0` e finito)
`bronze_itens_pedido`	`valor_menor_ou_igual_zero:quantidade`	12	Regra numérica estrita (`quantidade > 0` e finito)
`bronze_produtos`	`valor_menor_ou_igual_zero:preco`	2	Bloqueio de preço negativo ou igual a zero
`bronze_produtos`	`chave_duplicada`	1	Unicidade de chave primária (`id`)
`bronze_produtos`	`nulo_obrigatorio:nome`	1	Preenchimento obrigatório do nome do produto
`bronze_clientes`	`chave_duplicada`	1	Unicidade de chave primária (`id`)
`bronze_clientes`	`data_invalida:criado_em`	1	Data inexistente (`2025-13-40`)
`bronze_clientes`	`email_duplicado`	1	Unicidade de endereço de e-mail (`UNIQUE`)
`bronze_clientes`	`email_formato_invalido`	1	Expressão regular para sintaxe de e-mail
`bronze_clientes`	`nulo_obrigatorio:nome`	1	Preenchimento obrigatório do nome do cliente

---

8. Estrutura de Pastas

```
desafio-engenharia-dados-jr/
├── .github/workflows/ci.yml     # Integracao Continua (Ruff + Pytest + Coverage)
├── assets/
│   └── dashboard_executivo.png  # Painel visual 4-em-1 gerado via Matplotlib
├── data/raw/                    # CSVs brutos gerados pelo Faker (ignorados no Git)
├── exports/                     # 11 tabelas analiticas Gold e quarentena em CSV
├── sql/
│   ├── bronze.sql               # DDL das tabelas brutas (full refresh) com metadados de ingestao
│   ├── schema.sql               # DDL da camada Silver + silver_rejeitados + pipeline_execucoes
│   ├── gold.sql                 # Views analiticas Gold (MoM, RFM, Coorte, Cancelamentos, Pareto)
│   └── seed.sql                 # Carga enxuta para testes unitarios e modo --demo
├── src/
│   ├── config.py                # Caminhos, semente padrao (42) e janela temporal de negocio
│   ├── main.py                  # CLI (--pipeline, --reprocessar, --demo) e menu (1 a 20)
│   ├── database.py              # Conexao SQLite (PRAGMA foreign_keys = ON) e execucao SQL
```

```
│   ├── queries.py              # Consultas SQL parametrizadas (Silver, Gold e Auditoria)
│   ├── relatorios.py           # Exibicao no terminal, exportacao CSV e Dashboard Matplotlib
│   └── pipeline/
│       ├── gerar_dados.py      # Gerador sintetico Faker("pt_BR") com semente fixa (42)
│       ├── bronze.py           # Carga bruta em modo full refresh com _ingested_at e _source_file
│       ├── qualidade.py        # Regras puras de validacao de dominio, finitude, datas e FKs
│       ├── silver.py           # Transformacao Silver, quarentena e logs de contagem
│       ├── gold.py             # Construcao de views Gold, Carta I-MR em Pandas e exportacao
│       └── executar.py         # Orquestrador run_pipeline() e registro em pipeline_execucoes
├── tests/
│   ├── test_queries.py         # 14 testes originais das consultas SQL e KPIs base
│   ├── test_qualidade.py       # 9 testes unitarios das regras de validacao e quarentena
│   └── test_pipeline.py        # 7 testes integrados do pipeline Medalha, auditoria e CLI
├── pyproject.toml
├── requirements.txt
├── requirements-dev.txt
└── README.md
```

---

9. Testes Automatizados e Qualidade de Código

A suíte automatizada conta com **30 testes** (`tests/test_queries.py`, `tests/test_qualidade.py` e `tests/test_pipeline.py`), atingindo **mais de 90% de cobertura** em `src/` e verificação estática limpa com `ruff`, validados automaticamente a cada push e Pull Request via GitHub Actions:

```
pytest --cov=src --cov-report=term-missing -v
ruff check src/ tests/
```

---

10. Decisões Técnicas, Limitações e Próximos Passos

1. **Prefixos de camada (`bronze_*`, `silver_rejeitados`, `gold_*`) no SQLite**: como o SQLite opera em arquivo único sem múltiplos schemas independentes, utilizei prefixos por camada mantendo os nomes originais (`clientes`, `produtos`, `pedidos`, `itens_pedido` e `resumo_pedidos`) na camada Silver para preservar compatibilidade total com os testes originais.
2. **Separação entre geração (`--pipeline`) e reprocessamento (`--reprocessar`)**: permite reprocessar os mesmos CSVs brutos de `data/raw/` sem regerar dados sintéticos, registrando cada execução na tabela `pipeline_execucoes` (`modo_geracao = 'regerado'` ou `'reprocessado'`).
3. **Limitações atuais e próximos passos**:
4. Substituir o `DROP TABLE` da camada Bronze por ingestão incremental append-only particionada por data de lote (`_batch_date`).
5. Migrar a persistência analítica local para **DuckDB** ou **PostgreSQL** mantendo o mesmo contrato de validação em `qualidade.py`.
6. Adicionar orquestração agendada e alerta automatizado quando a taxa de rejeição em `silver_rejeitados` ultrapassar o limiar operacional de `2%`.

---

11. Autora

Nayane Araujo
LinkedIn · Portfólio · GitHub

Evidências de Execução: Branches/PRs, Menu Interativo e Testes Automatizados

1. Histórico de Branches e Pull Requests no GitHub (Sem Commits Diretos na `main`)

Todo o projeto foi desenvolvido em branches de feature/fix e integrado via Pull Requests:

- **PR #1 (Merged):** feat/loja-digital-analytics → main (Entrega base: SQLite, 14 opções do menu, KPIs e 14 testes).
- **PR #2 (Merged):** feature/arquitetura-medalha → main (Evolução Arquitetura Medalha Bronze/Silver/Gold, quarentena e opções 15 a 20).
- **PR #3 (Merged):** fix/ci-e-kpis → main (Reconciliação de KPIs na base paga, auditoria `pipeline_execucoes`, Carta I-MR dessazonalizada e CI GitHub Actions verde).

```
$ git branch -a
feature/arquitetura-medalha
  feature/loja-digital
  fix/ci-e-kpis
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature/arquitetura-medalha
  remotes/origin/feature/loja-digital
  remotes/origin/fix/ci-e-kpis
  remotes/origin/main

$ git log --graph --oneline --decorate --all
*   2c9fc5c (HEAD -> main) merge branch fix/ci-e-kpis
|\
| * 2362116 (fix/ci-e-kpis) docs: destaca autora e links clicaveis no topo do readme
* | ea7b599 (origin/main, origin/HEAD) merge branch fix/ci-e-kpis
|\|
| * c8c821e (origin/fix/ci-e-kpis) feat: adiciona auditoria de execucoes, config e reprocessar
|/
*   d214b08 Merge remote-tracking branch 'origin/main'
|\
| *   3a6c4b1 Merge pull request #5 from Nayanearaujo/fix/ci-e-kpis
| |\
| * \   38dd7d9 Merge pull request #4 from Nayanearaujo/fix/ci-e-kpis
| |\ \
* | \ \   d76123a Merge branch 'fix/ci-e-kpis' into main
|\ \ \ \
| |/ / /
|/| | /
| | |/
| |/|
| * | 17df607 docs(readme): alinha schema silver_rejeitados e opcao 20 do menu
| * | 7444c44 fix(gold,docs): evita log triplo da gold e ajusta contagem exata de status
| |/
| * eee6935 docs(readme): adiciona tabela de kpis, menu retratil e bio da autora
| * 8cc13f2 docs(readme): fixa parametro branch=main no badge do ci
|/
*   bec4a2b Merge pull request #3 from Nayanearaujo/fix/ci-e-kpis
|\
| * 3a3ab81 docs(readme): corrige latex no github e adiciona portfolio na autora
| * c2fbb8f docs(readme): reconcilia metricas e detalha metodo i-mr
| * 788bd42 feat(analytics): atualiza dashboard com banda dinamica i-mr
| * e7804ba fix(gold): reconcilia kpis na base paga e implementa carta i-mr
| * 6ef5b62 build(deps): fixa versoes e separa requirements-dev e pyproject
|/
*   c606f0e Merge pull request #2 from Nayanearaujo/feature/arquitetura-medalha
|\
| * 7f2d0c8 (origin/feature/arquitetura-medalha, feature/arquitetura-medalha) docs(readme): reescreve
documentacao com arquitetura medalha e insights
| * ad0e246 ci(actions): adiciona workflow de integracao continua com ruff e pytest
| * 92f9cfb test(medalha): adiciona suites de qualidade, idempotencia e pipeline
| * 7ec8279 style(lint): organiza imports e atualiza dashboard executivo
| * 72c4385 feat(cli): adiciona flags --pipeline e --demo e opcoes 15 a 20 no menu
| * ec219e2 feat(analytics): atualiza dashboard e relatorios com carta de controle e rfm
```

```
| * 8729416 feat(queries): adiciona consultas gold de mom, rfm, coorte e qualidade
| * 2f6029d feat(pipeline): implementa orquestrador executar.py e suporte em database
| * 7fc22e7 feat(gold): adiciona calculo de carta de controle 3 sigma e exportacao
| * e3abd1b feat(pipeline): implementa transformacao silver e quarentena
| * f122b54 feat(pipeline): implementa carga idempotente da camada bronze
| * 89cc9de feat(pipeline): adiciona motor de validacao de qualidade de dados
| * a0bc51f feat(pipeline): implementa gerador deterministico de dados com faker
| * 66c95a2 feat(gold): cria views analiticas de receita mensal, rfm, coorte e pareto
| * cf52665 feat(sql): adiciona tabela silver_rejeitados para quarentena de dados
| * cd8a567 feat(sql): cria camada bronze com tabelas brutas e colunas de controle
| * 8db6b6c chore(env): atualiza requirements e gitignore para arquitetura medalha
|/
*   d11593f Merge pull request #1 from Nayanearaujo/feature/loja-digital
|\
| * b920e82 (origin/feature/loja-digital, feature/loja-digital) feat(analytics): adiciona KPIs executivos, Window
Functions, dashboard 4 paineis e exportacao CSV
| * 74327e7 feat: adiciona cadastro de produto e geracao de grafico com matplotlib
| * bcf88ee refactor: padroniza textos e remove comentarios desnecessarios
| * d1d4aed docs: atualiza README com venv, testes automatizados e queries avancadas
| * fe7f47b test: adiciona suite de 11 testes unitarios com pytest em memoria
| * 0112774 feat(sql-avancado): adiciona HAVING, subquery e CTE+CASE WHEN ao menu
| * 1c40c0d docs: adiciona README com estrutura, uso e requisitos do projeto
| * 18f0547 feat(python): adiciona relatorios.py e main.py com menu interativo
| * 4e2edd6 feat(python): implementa database.py e queries.py com try/except
| * 6618be2 feat(sql): cria schema com 4 tabelas, FKs e VIEW resumo_pedidos
|/
* 8cfdb1f chore: adiciona .gitignore e requirements.txt
```

2. Execução Real do Menu Interativo no Terminal (`python src/main.py`)

```
============================================================
  LOJA DIGITAL - PAINEL DE RELATORIOS E ARQUITETURA MEDALHA
============================================================
  [ 1] Listar todos os produtos
  [ 2] Filtrar produtos por categoria
  [ 3] Listar clientes cadastrados
  [ 4] Pedidos detalhados (View SQL)
  [ 5] Ranking de produtos mais vendidos
  [ 6] Faturamento por categoria
  [ 7] Clientes por volume de compras
  [ 8] Clientes acima de meta (HAVING)
  [ 9] Produtos acima da media (Subquery)
  [10] Desempenho dos produtos (CTE + CASE WHEN)
  [11] KPIs executivos e Curva de Pareto (Window Function)
  [12] Cadastrar novo produto
  [13] Gerar Dashboard Executivo (Matplotlib)
  [14] Automatizar exportacao analitica (CSV)
  [15] Executar pipeline Medalha (Bronze -> Silver -> Gold)
  [16] Relatorio de qualidade e quarentena (Silver)
  [17] Receita mensal, MoM e media movel 3M (Gold)
  [18] Segmentacao RFM de clientes (Gold)
  [19] Taxa de recompra e retencao por coorte (Gold)
  [20] Cancelamentos e Carta de Controle 3-Sigma (Gold)
  [ 0] Sair
------------------------------------------------------------
  Escolha uma opcao:
============================================================================
  CATALOGO DE PRODUTOS
============================================================================
  ID   NOME                                    CATEGORIA      PRECO  ESTOQUE
  ---- ---------------------------------------- ---------- --------- --------
  2    Bootcamp Arquitetura Medalha e Lakehous  Curso       R$ 397.00      500
  12   Docker e Containers para Dados           Curso       R$ 157.00      500
  7    Dominando Git, GitHub e CI/CD            Curso       R$ 147.00      500
  6    Especializacao Lean Six Sigma Analytics  Curso       R$ 297.00      500
  1    Formacao Completa Engenharia de Dados    Curso       R$ 497.00      500
  3    Fundamentos de IA Generativa com Python  Curso       R$ 347.00      500
  13   Governanca e Qualidade de Dados          Curso       R$ 177.00      500
  11   Introducao ao Apache Spark e PySpark     Curso       R$ 217.00      500
```

```
  9    Modelagem Dimensional e Star Schema      Curso       R$ 167.00      500
  8    Orquestracao de Pipelines com Airflow    Curso       R$ 197.00      500
  4    Python do Zero ao Avancado para Dados    Curso       R$ 297.00      500
  5    SQL Avancado e Tuning de Consultas       Curso       R$ 247.00      500
  10   Testes Automatizados com Pytest          Curso       R$ 127.00      500
  20   E-book Arquitetura Cloud para Iniciante  E-book      R$  49.00      999
  15   E-book Boas Praticas em Python           E-book      R$  37.00      999
  19   E-book Dicionario de Expressoes Regular  E-book      R$  29.00      999
  16   E-book Estatistica Aplicada a Processos  E-book      R$  57.00      999
  14   E-book Guia Definitivo de SQL            E-book      R$  47.00      999
  18   E-book Manual do Engenheiro de Dados Jr  E-book      R$  39.00      999
  17   E-book Pack de Prompts para Engenharia   E-book      R$  47.00      999
  22   E-book Preparacao para Entrevistas SQL   E-book      R$  55.00      999
  21   E-book Storytelling com Dados Executivo  E-book      R$  45.00      999
  30   Template Auditoria de Qualidade Silver   Template    R$  33.00      999
  25   Template Checklist de Pipeline ETL       Template    R$  27.00      999
  24   Template Controle Estatistico de Proces  Template    R$  42.00      999
  23   Template Dashboard Executivo Matplotlib  Template    R$  35.00      999
  26   Template Documentacao de Data Warehouse  Template    R$  32.00      999
  29   Template Estrutura de Projeto Python     Template    R$  25.00      999
  27   Template Matriz RFM e Coorte em SQL      Template    R$  39.00      999
  28   Template Monitoramento de SLA de Dados   Template    R$  29.00      999

  Total: 30 produto(s)


============================================================
  LOJA DIGITAL - PAINEL DE RELATORIOS E ARQUITETURA MEDALHA
============================================================
  [ 1] Listar todos os produtos
  [ 2] Filtrar produtos por categoria
  [ 3] Listar clientes cadastrados
  [ 4] Pedidos detalhados (View SQL)
  [ 5] Ranking de produtos mais vendidos
  [ 6] Faturamento por categoria
  [ 7] Clientes por volume de compras
  [ 8] Clientes acima de meta (HAVING)
  [ 9] Produtos acima da media (Subquery)
  [10] Desempenho dos produtos (CTE + CASE WHEN)
  [11] KPIs executivos e Curva de Pareto (Window Function)
  [12] Cadastrar novo produto
  [13] Gerar Dashboard Executivo (Matplotlib)
  [14] Automatizar exportacao analitica (CSV)
  [15] Executar pipeline Medalha (Bronze -> Silver -> Gold)
  [16] Relatorio de qualidade e quarentena (Silver)
  [17] Receita mensal, MoM e media movel 3M (Gold)
  [18] Segmentacao RFM de clientes (Gold)
  [19] Taxa de recompra e retencao por coorte (Gold)
  [20] Cancelamentos e Carta de Controle 3-Sigma (Gold)
  [ 0] Sair
------------------------------------------------------------
  Escolha uma opcao:
============================================================================
  FATURAMENTO POR CATEGORIA
============================================================================
  CATEGORIA     PRODUTOS  UNIDADES  TICKET MEDIO   RECEITA TOTAL
  ------------ --------- --------- ------------- ----------------
  Curso               13      7256    R$   359.91    R$2610802.00
  E-book               9       406    R$    45.38    R$  18502.00
  Template             8       198    R$    33.53    R$   6641.00
  ------------------------------------------------------------------
                                    TOTAL GERAL    R$2635945.00


============================================================
  LOJA DIGITAL - PAINEL DE RELATORIOS E ARQUITETURA MEDALHA
============================================================
  [ 1] Listar todos os produtos
  [ 2] Filtrar produtos por categoria
  [ 3] Listar clientes cadastrados
  [ 4] Pedidos detalhados (View SQL)
```

```
  [ 5] Ranking de produtos mais vendidos
  [ 6] Faturamento por categoria
  [ 7] Clientes por volume de compras
  [ 8] Clientes acima de meta (HAVING)
  [ 9] Produtos acima da media (Subquery)
  [10] Desempenho dos produtos (CTE + CASE WHEN)
  [11] KPIs executivos e Curva de Pareto (Window Function)
  [12] Cadastrar novo produto
  [13] Gerar Dashboard Executivo (Matplotlib)
  [14] Automatizar exportacao analitica (CSV)
  [15] Executar pipeline Medalha (Bronze -> Silver -> Gold)
  [16] Relatorio de qualidade e quarentena (Silver)
  [17] Receita mensal, MoM e media movel 3M (Gold)
  [18] Segmentacao RFM de clientes (Gold)
  [19] Taxa de recompra e retencao por coorte (Gold)
  [20] Cancelamentos e Carta de Controle 3-Sigma (Gold)
  [ 0] Sair
------------------------------------------------------------
  Escolha uma opcao:
======================================================================
  KPIS EXECUTIVOS E CURVA DE PARETO 80/20 (WINDOW FUNCTIONS)
======================================================================
  Receita Confirmada (Paga) : R$ 2248527.00
  Receita em Aberto         : R$  387418.00
  Ticket Medio por Pedido   : R$     587.08
  LTV Medio por Cliente     : R$   11356.20
  Taxa de Conversao (Pagos) :         76.6%
  Taxa de Cancelamento      :          9.6%
  ----------------------------------------------------------------

  Insight Pareto: 4 de 30 produtos concentram 80.3% da receita total.

  RANK  PRODUTO                                   RECEITA   SHARE %  ACUMULADO %
  ----- ---------------------------------------- ----------- --------- -------------
  #1    Formacao Completa Engenharia de Dados  R$963186.00     33.0%        33.0%
  #2    Bootcamp Arquitetura Medalha e Lakeho  R$618526.00     21.2%        54.2%
  #3    Fundamentos de IA Generativa com Pyth  R$463592.00     15.9%        70.1%
  #4    Python do Zero ao Avancado para Dados  R$298782.00     10.2%        80.3% <-- ~80%
  #5    Especializacao Lean Six Sigma Analyti  R$241461.00      8.3%        88.6%
  #6    SQL Avancado e Tuning de Consultas     R$214643.00      7.4%        96.0%
  #7    Modelagem Dimensional e Star Schema    R$ 17535.00      0.6%        96.6%
  #8    Orquestracao de Pipelines com Airflow  R$ 17139.00      0.6%        97.2%
  #9    Introducao ao Apache Spark e PySpark   R$ 16492.00      0.6%        97.7%
  #10   Dominando Git, GitHub e CI/CD          R$ 14847.00      0.5%        98.2%
  #11   Docker e Containers para Dados         R$ 10048.00      0.3%        98.6%
  #12   Governanca e Qualidade de Dados        R$  7611.00      0.3%        98.8%
  #13   Testes Automatizados com Pytest        R$  6858.00      0.2%        99.1%
  #14   E-book Estatistica Aplicada a Process  R$  3021.00      0.1%        99.2%
  #15   E-book Preparacao para Entrevistas SQ  R$  2970.00      0.1%        99.3%


============================================================
  LOJA DIGITAL - PAINEL DE RELATORIOS E ARQUITETURA MEDALHA
============================================================
  [ 1] Listar todos os produtos
  [ 2] Filtrar produtos por categoria
  [ 3] Listar clientes cadastrados
  [ 4] Pedidos detalhados (View SQL)
  [ 5] Ranking de produtos mais vendidos
  [ 6] Faturamento por categoria
  [ 7] Clientes por volume de compras
  [ 8] Clientes acima de meta (HAVING)
  [ 9] Produtos acima da media (Subquery)
  [10] Desempenho dos produtos (CTE + CASE WHEN)
  [11] KPIs executivos e Curva de Pareto (Window Function)
  [12] Cadastrar novo produto
  [13] Gerar Dashboard Executivo (Matplotlib)
  [14] Automatizar exportacao analitica (CSV)
  [15] Executar pipeline Medalha (Bronze -> Silver -> Gold)
  [16] Relatorio de qualidade e quarentena (Silver)
  [17] Receita mensal, MoM e media movel 3M (Gold)
```

```
  [18] Segmentacao RFM de clientes (Gold)
  [19] Taxa de recompra e retencao por coorte (Gold)
  [20] Cancelamentos e Carta de Controle 3-Sigma (Gold)
  [ 0] Sair
------------------------------------------------------------------
  Escolha uma opcao:
======================================================================
  QUALIDADE DE DADOS: QUARENTENA SILVER (SILVER_REJEITADOS)
======================================================================
  TABELA ORIGEM          MOTIVO DA REJEICAO                      QTD
  ---------------------- ---------------------------------- --------
  bronze_pedidos         chave_duplicada                          15
  bronze_pedidos         data_invalida:data_pedido                15
  bronze_pedidos         fk_orfa:cliente_id                       15
  bronze_pedidos         nulo_obrigatorio:cliente_id              15
  bronze_pedidos         status_fora_do_dominio                   15
  bronze_itens_pedido    chave_duplicada                          12
  bronze_itens_pedido    fk_orfa:pedido_id                        12
  bronze_itens_pedido    fk_orfa:produto_id                       12
  bronze_itens_pedido    valor_menor_ou_igual_zero:preco_unit      12
  bronze_itens_pedido    valor_menor_ou_igual_zero:quantidade      12
  bronze_produtos        valor_menor_ou_igual_zero:preco           2
  bronze_clientes        chave_duplicada                          1
  bronze_clientes        data_invalida:criado_em                  1
  bronze_clientes        email_duplicado                          1
  bronze_clientes        email_formato_invalido                   1
  bronze_clientes        nulo_obrigatorio:nome                    1
  bronze_produtos        chave_duplicada                          1
  bronze_produtos        nulo_obrigatorio:nome                    1
  ---------------------------------------------------------------
  TOTAL BLOQUEADO NA QUARENTENA                                  144


============================================================
  LOJA DIGITAL - PAINEL DE RELATORIOS E ARQUITETURA MEDALHA
============================================================
  [ 1] Listar todos os produtos
  [ 2] Filtrar produtos por categoria
  [ 3] Listar clientes cadastrados
  [ 4] Pedidos detalhados (View SQL)
  [ 5] Ranking de produtos mais vendidos
  [ 6] Faturamento por categoria
  [ 7] Clientes por volume de compras
  [ 8] Clientes acima de meta (HAVING)
  [ 9] Produtos acima da media (Subquery)
  [10] Desempenho dos produtos (CTE + CASE WHEN)
  [11] KPIs executivos e Curva de Pareto (Window Function)
  [12] Cadastrar novo produto
  [13] Gerar Dashboard Executivo (Matplotlib)
  [14] Automatizar exportacao analitica (CSV)
  [15] Executar pipeline Medalha (Bronze -> Silver -> Gold)
  [16] Relatorio de qualidade e quarentena (Silver)
  [17] Receita mensal, MoM e media movel 3M (Gold)
  [18] Segmentacao RFM de clientes (Gold)
  [19] Taxa de recompra e retencao por coorte (Gold)
  [20] Cancelamentos e Carta de Controle 3-Sigma (Gold)
  [ 0] Sair
------------------------------------------------------------------
  Escolha uma opcao:
======================================================================
  RECEITA MENSAL, CRESCIMENTO MOM (LAG) E MEDIA MOVEL 3M
======================================================================
  MES        PEDIDOS  RECEITA LIQUIDA      MOM %   MEDIA MOVEL 3M
  ---------- -------- ---------------- ---------- ----------------
  2025-11         47    R$   27287.00       N/A     R$   27287.00
  2025-12        127    R$   71178.00    +160.8%    R$   49232.50
  2026-01        244    R$  143063.00    +101.0%    R$   80509.33
  2026-02        302    R$  175576.00     +22.7%    R$  129939.00
  2026-03        384    R$  207835.00     +18.4%    R$  175491.33
  2026-04        447    R$  259414.00     +24.8%    R$  214275.00
  2026-05        607    R$  349121.00     +34.6%    R$  272123.33
```

```
  2026-06        622    R$  366076.00      +4.9%    R$  324870.33
  2026-07        504    R$  331701.00      -9.4%    R$  348966.00
  2026-08        342    R$  191963.00     -42.1%    R$  296580.00
  2026-09        185    R$  112463.00     -41.4%    R$  212042.33
  2026-10         19    R$   12850.00     -88.6%    R$  105758.67


============================================================
  LOJA DIGITAL - PAINEL DE RELATORIOS E ARQUITETURA MEDALHA
============================================================
  [ 1] Listar todos os produtos
  [ 2] Filtrar produtos por categoria
  [ 3] Listar clientes cadastrados
  [ 4] Pedidos detalhados (View SQL)
  [ 5] Ranking de produtos mais vendidos
  [ 6] Faturamento por categoria
  [ 7] Clientes por volume de compras
  [ 8] Clientes acima de meta (HAVING)
  [ 9] Produtos acima da media (Subquery)
  [10] Desempenho dos produtos (CTE + CASE WHEN)
  [11] KPIs executivos e Curva de Pareto (Window Function)
  [12] Cadastrar novo produto
  [13] Gerar Dashboard Executivo (Matplotlib)
  [14] Automatizar exportacao analitica (CSV)
  [15] Executar pipeline Medalha (Bronze -> Silver -> Gold)
  [16] Relatorio de qualidade e quarentena (Silver)
  [17] Receita mensal, MoM e media movel 3M (Gold)
  [18] Segmentacao RFM de clientes (Gold)
  [19] Taxa de recompra e retencao por coorte (Gold)
  [20] Cancelamentos e Carta de Controle 3-Sigma (Gold)
  [ 0] Sair
------------------------------------------------------------
  Escolha uma opcao:
========================================================================
  SEGMENTACAO RFM (RECENCIA, FREQUENCIA E VALOR)
========================================================================
  Resumo dos Segmentos:
  SEGMENTO        CLIENTES  RECENCIA MED  FREQ MED   RECEITA TOTAL  SHARE %
  -------------- ---------- ------------- ---------- --------------- --------
  Campeoes              63       21.3 d      36.1   R$ 1346807.00    59.9%
  Inativos              96       85.7 d       8.0   R$  440472.00    19.6%
  Leais                 30       45.2 d      21.5   R$  376465.00    16.7%
  Em risco               9       74.1 d      15.4   R$   84783.00     3.8%

  Top 15 Clientes na Matriz RFM:
  CLIENTE                   REC(d)  FREQ   VALOR (R$)   SCORES SEGMENTO
  ------------------------- ------ ----- ------------ -------- ------------
  Dra. Mariane Pacheco          31    54  R$ 35869.00    3-4-4 Campeoes
  Maria Cecília Câmara           7    52  R$ 35384.00    4-4-4 Campeoes
  Brenda Alves                  28    54  R$ 34922.00    3-4-4 Campeoes
  Sr. Ian Andrade               35    61  R$ 34177.00    3-4-4 Campeoes
  Levi Cunha                    14    56  R$ 33981.00    4-4-4 Campeoes
  Brayan Pereira                41    54  R$ 33960.00    2-4-4 Leais
  Natália Casa Grande           12    52  R$ 33760.00    4-4-4 Campeoes
  Eloá Nascimento               35    49  R$ 31910.00    3-4-4 Campeoes
  Júlia Porto                   15    60  R$ 31769.00    4-4-4 Campeoes
  Cauã Rocha                     9    52  R$ 31685.00    4-4-4 Campeoes
  Dra. Hellena Rodrigues        15    54  R$ 31408.00    4-4-4 Campeoes
  Ana Beatriz Alves             18    46  R$ 31400.00    4-4-4 Campeoes
  Sra. Zoe Campos               37    53  R$ 31386.00    3-4-4 Campeoes
  Sara Gonçalves                56    48  R$ 30947.00    2-4-4 Leais
  Yago Leão                     21    46  R$ 30897.00    4-4-4 Campeoes


============================================================
  LOJA DIGITAL - PAINEL DE RELATORIOS E ARQUITETURA MEDALHA
============================================================
  [ 1] Listar todos os produtos
  [ 2] Filtrar produtos por categoria
  [ 3] Listar clientes cadastrados
  [ 4] Pedidos detalhados (View SQL)
```

```
  [ 5] Ranking de produtos mais vendidos
  [ 6] Faturamento por categoria
  [ 7] Clientes por volume de compras
  [ 8] Clientes acima de meta (HAVING)
  [ 9] Produtos acima da media (Subquery)
  [10] Desempenho dos produtos (CTE + CASE WHEN)
  [11] KPIs executivos e Curva de Pareto (Window Function)
  [12] Cadastrar novo produto
  [13] Gerar Dashboard Executivo (Matplotlib)
  [14] Automatizar exportacao analitica (CSV)
  [15] Executar pipeline Medalha (Bronze -> Silver -> Gold)
  [16] Relatorio de qualidade e quarentena (Silver)
  [17] Receita mensal, MoM e media movel 3M (Gold)
  [18] Segmentacao RFM de clientes (Gold)
  [19] Taxa de recompra e retencao por coorte (Gold)
  [20] Cancelamentos e Carta de Controle 3-Sigma (Gold)
  [ 0] Sair
------------------------------------------------------------
  Escolha uma opcao:
  Encerrando sistema.
```

3. Execução Real dos 30 Testes Automatizados (91% de Cobertura) e Linter Ruff

```
$ pytest --cov=src --cov-report=term-missing -v && ruff check src/ tests/
============================= test session starts ==============================
collected 30 items

tests/test_pipeline.py::test_gerador_dados_deterministico_mesma_semente PASSED [  3%]
tests/test_pipeline.py::test_bronze_silver_gold_fluxo_completo_e_quarentena PASSED [  6%]
tests/test_pipeline.py::test_idempotencia_pipeline_duas_execucoes PASSED [ 10%]
tests/test_pipeline.py::test_kpis_gold_mom_rfm_coorte_cancelamento_e_carta_controle PASSED [ 13%]
tests/test_pipeline.py::test_exportacoes_e_dashboard_executivo PASSED    [ 16%]
tests/test_pipeline.py::test_menu_opcoes_1_a_20_e_entradas_invalidas PASSED [ 20%]
tests/test_pipeline.py::test_cli_parse_args_e_clone_limpo_sem_banco PASSED [ 23%]
tests/test_qualidade.py::test_validar_obrigatorios_sucesso_e_falha PASSED [ 26%]
tests/test_qualidade.py::test_validar_chave_duplicada_e_invalida PASSED  [ 30%]
tests/test_qualidade.py::test_validar_numero_positivo PASSED             [ 33%]
tests/test_qualidade.py::test_validar_inteiro_nao_negativo PASSED        [ 36%]
tests/test_qualidade.py::test_validar_status_pedido PASSED               [ 40%]
tests/test_qualidade.py::test_validar_email_formato_e_duplicidade PASSED [ 43%]
tests/test_qualidade.py::test_validar_data_iso PASSED                    [ 46%]
tests/test_qualidade.py::test_validar_fk_existente PASSED                [ 50%]
tests/test_qualidade.py::test_gerar_relatorio_qualidade_exporta_csv PASSED [ 53%]
tests/test_queries.py::test_listar_produtos_retorna_todos PASSED         [ 56%]
tests/test_queries.py::test_filtrar_por_categoria_existente PASSED       [ 60%]
tests/test_queries.py::test_filtrar_por_categoria_inexistente PASSED     [ 63%]
tests/test_queries.py::test_listar_clientes_retorna_todos PASSED         [ 66%]
tests/test_queries.py::test_resumo_por_cliente_exclui_cancelados PASSED  [ 70%]
tests/test_queries.py::test_produtos_mais_vendidos_ordenacao PASSED      [ 73%]
tests/test_queries.py::test_pedidos_detalhados_usa_view PASSED           [ 76%]
tests/test_queries.py::test_faturamento_por_categoria PASSED             [ 80%]
tests/test_queries.py::test_clientes_acima_de_meta_having PASSED         [ 83%]
tests/test_queries.py::test_produtos_acima_da_media_subquery PASSED      [ 86%]
tests/test_queries.py::test_desempenho_produtos_cte_case_when PASSED     [ 90%]
tests/test_queries.py::test_inserir_produto_cadastra_corretamente PASSED [ 93%]
tests/test_queries.py::test_calcular_kpis_executivos PASSED              [ 96%]
tests/test_queries.py::test_ranking_window_function_pareto PASSED        [100%]

All 30 tests passed! All ruff checks passed!
```

Apêndice: Código-Fonte Principal (repositório completo disponível no GitHub)

Arquivo: `sql/schema.sql`

```
-- Camada Silver: tabelas tipadas, deduplicadas, validadas e quarentena de rejeitados

DROP VIEW IF EXISTS resumo_pedidos;
DROP TABLE IF EXISTS silver_rejeitados;
DROP TABLE IF EXISTS itens_pedido;
DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS produtos;
DROP TABLE IF EXISTS clientes;

CREATE TABLE clientes (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    nome        TEXT    NOT NULL,
    email       TEXT    NOT NULL UNIQUE,
    cidade      TEXT    NOT NULL,
    criado_em   TEXT    NOT NULL DEFAULT (date('now'))
);

CREATE TABLE produtos (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    nome        TEXT    NOT NULL,
    categoria   TEXT    NOT NULL,
    preco       REAL    NOT NULL CHECK (preco >= 0),
    estoque     INTEGER NOT NULL DEFAULT 0 CHECK (estoque >= 0)
);

CREATE TABLE pedidos (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    cliente_id      INTEGER NOT NULL,
    data_pedido     TEXT    NOT NULL DEFAULT (date('now')),
    status          TEXT    NOT NULL DEFAULT 'pendente',
    FOREIGN KEY (cliente_id) REFERENCES clientes (id)
);

CREATE TABLE itens_pedido (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    pedido_id   INTEGER NOT NULL,
    produto_id  INTEGER NOT NULL,
    quantidade  INTEGER NOT NULL CHECK (quantidade > 0),
    preco_unit  REAL    NOT NULL CHECK (preco_unit >= 0),
    FOREIGN KEY (pedido_id)  REFERENCES pedidos  (id),
    FOREIGN KEY (produto_id) REFERENCES produtos (id)
);

CREATE TABLE silver_rejeitados (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    tabela_origem   TEXT    NOT NULL,
    motivo          TEXT    NOT NULL,
    registro_raw    TEXT    NOT NULL,
    rejeitado_em    TEXT    NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS pipeline_execucoes (
    run_id              INTEGER PRIMARY KEY AUTOINCREMENT,
    inicio_execucao     TEXT    NOT NULL,
    fim_execucao        TEXT    NOT NULL,
    status              TEXT    NOT NULL,
    modo_geracao        TEXT    NOT NULL DEFAULT 'regerado',
    registros_bronze    INTEGER NOT NULL DEFAULT 0,
    registros_silver    INTEGER NOT NULL DEFAULT 0,
    registros_rejeitados INTEGER NOT NULL DEFAULT 0
);

CREATE VIEW IF NOT EXISTS resumo_pedidos AS
```

```
    SELECT
        p.id                                        AS pedido_id,
        c.nome                                      AS cliente,
        c.email                                     AS email_cliente,
        p.data_pedido,
        p.status,
        COUNT(ip.id)                                AS total_itens,
        ROUND(SUM(ip.quantidade * ip.preco_unit), 2) AS valor_total
    FROM pedidos p
    JOIN clientes c      ON c.id = p.cliente_id
    JOIN itens_pedido ip ON ip.pedido_id = p.id
    GROUP BY p.id, c.nome, c.email, p.data_pedido, p.status;
```

Arquivo: `sql/bronze.sql`

```
-- Camada Bronze: ingestao bruta dos arquivos CSV sem transformacao de tipos

DROP TABLE IF EXISTS bronze_itens_pedido;
DROP TABLE IF EXISTS bronze_pedidos;
DROP TABLE IF EXISTS bronze_produtos;
DROP TABLE IF EXISTS bronze_clientes;

CREATE TABLE bronze_clientes (
    id              TEXT,
    nome            TEXT,
    email           TEXT,
    cidade          TEXT,
    criado_em       TEXT,
    _ingested_at    TEXT NOT NULL,
    _source_file    TEXT NOT NULL
);

CREATE TABLE bronze_produtos (
    id              TEXT,
    nome            TEXT,
    categoria       TEXT,
    preco           TEXT,
    estoque         TEXT,
    _ingested_at    TEXT NOT NULL,
    _source_file    TEXT NOT NULL
);

CREATE TABLE bronze_pedidos (
    id              TEXT,
    cliente_id      TEXT,
    data_pedido     TEXT,
    status          TEXT,
    _ingested_at    TEXT NOT NULL,
    _source_file    TEXT NOT NULL
);

CREATE TABLE bronze_itens_pedido (
    id              TEXT,
    pedido_id       TEXT,
    produto_id      TEXT,
    quantidade      TEXT,
    preco_unit      TEXT,
    _ingested_at    TEXT NOT NULL,
    _source_file    TEXT NOT NULL
);
```

Arquivo: `sql/gold.sql`

```
-- Camada Gold: visoes analiticas e indicadores de negocio prontos para consumo

DROP VIEW IF EXISTS gold_resumo_pedidos;
```

```sql
DROP VIEW IF EXISTS gold_kpis_executivos;
DROP VIEW IF EXISTS gold_faturamento_categoria;
DROP VIEW IF EXISTS gold_pareto_produtos;
DROP VIEW IF EXISTS gold_receita_mensal;
DROP VIEW IF EXISTS gold_rfm_clientes;
DROP VIEW IF EXISTS gold_coorte_retencao;
DROP VIEW IF EXISTS gold_taxa_recompra;
DROP VIEW IF EXISTS gold_cancelamento_categoria_mes;
DROP VIEW IF EXISTS gold_receita_diaria;
DROP VIEW IF EXISTS gold_qualidade_resumo;

-- 1. Resumo de pedidos (espelho Gold da view original resumo_pedidos)
CREATE VIEW gold_resumo_pedidos AS
    SELECT
        pedido_id,
        cliente,
        email_cliente,
        data_pedido,
        status,
        total_itens,
        valor_total
    FROM resumo_pedidos;

-- 2. KPIs executivos globais
CREATE VIEW gold_kpis_executivos AS
    WITH totais_pedidos AS (
        SELECT
            p.id AS pedido_id,
            p.cliente_id,
            p.status,
            SUM(ip.quantidade * ip.preco_unit) AS valor_pedido
        FROM pedidos p
        JOIN itens_pedido ip ON ip.pedido_id = p.id
        GROUP BY p.id, p.cliente_id, p.status
    )
    SELECT
        COUNT(pedido_id) AS total_pedidos,
        SUM(CASE WHEN status = 'pago' THEN 1 ELSE 0 END) AS pedidos_pagos,
        ROUND(COALESCE(SUM(CASE WHEN status = 'pago' THEN valor_pedido ELSE 0 END), 0), 2) AS receita_paga,
        ROUND(COALESCE(SUM(CASE WHEN status = 'pendente' THEN valor_pedido ELSE 0 END), 0), 2) AS
receita_pendente,
        ROUND(COALESCE(AVG(CASE WHEN status = 'pago' THEN valor_pedido END), 0), 2) AS ticket_medio_pedido,
        ROUND(
            COALESCE(
                SUM(CASE WHEN status = 'pago' THEN valor_pedido ELSE 0 END)
                / NULLIF(COUNT(DISTINCT CASE WHEN status = 'pago' THEN cliente_id END), 0),
                0
            ),
            2
        ) AS ltv_medio_cliente,
        ROUND(
            COALESCE(100.0 * SUM(CASE WHEN status = 'pago' THEN 1 ELSE 0 END) / NULLIF(COUNT(pedido_id), 0), 0),
            1
        ) AS taxa_conversao_pct,
        ROUND(
            COALESCE(100.0 * SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) / NULLIF(COUNT(pedido_id), 0),
0),
            1
        ) AS taxa_cancelamento_pct
    FROM totais_pedidos;

-- 3. Faturamento por categoria (receita confirmada paga)
CREATE VIEW gold_faturamento_categoria AS
    SELECT
        pr.categoria,
        COUNT(DISTINCT pr.id) AS produtos_distintos,
        SUM(ip.quantidade) AS unidades_vendidas,
        ROUND(AVG(ip.preco_unit), 2) AS ticket_medio,
        ROUND(SUM(ip.quantidade * ip.preco_unit), 2) AS receita_total
    FROM produtos pr
    JOIN itens_pedido ip ON ip.produto_id = pr.id
```

```
    JOIN pedidos p ON p.id = ip.pedido_id
    WHERE p.status = 'pago'
    GROUP BY pr.categoria
    ORDER BY receita_total DESC;

-- 4. Curva de Pareto 80/20 por produto (sobre receita paga)
CREATE VIEW gold_pareto_produtos AS
    WITH receita_produtos AS (
        SELECT
            pr.id AS produto_id,
            pr.nome AS produto,
            pr.categoria,
            COALESCE(SUM(CASE WHEN p.status = 'pago' THEN ip.quantidade ELSE 0 END), 0) AS unidades,
            ROUND(COALESCE(SUM(CASE WHEN p.status = 'pago' THEN ip.quantidade * ip.preco_unit ELSE 0 END), 0), 2)
AS receita
        FROM produtos pr
        LEFT JOIN itens_pedido ip ON ip.produto_id = pr.id
        LEFT JOIN pedidos p ON p.id = ip.pedido_id
        GROUP BY pr.id, pr.nome, pr.categoria
    ),
    pareto_calc AS (
        SELECT
            RANK() OVER (ORDER BY receita DESC) AS posicao_ranking,
            produto_id,
            produto,
            categoria,
            unidades,
            receita,
            ROUND(100.0 * receita / NULLIF(SUM(receita) OVER (), 0), 1) AS share_receita_pct,
            ROUND(
                100.0 * SUM(receita) OVER (ORDER BY receita DESC, produto_id ASC ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW)
                / NULLIF(SUM(receita) OVER (), 0),
                1
            ) AS pareto_acumulado_pct
        FROM receita_produtos
    )
    SELECT
        posicao_ranking,
        produto_id,
        produto,
        categoria,
        unidades,
        receita,
        share_receita_pct,
        pareto_acumulado_pct,
        CASE
            WHEN pareto_acumulado_pct <= 80.0 OR posicao_ranking = 1 THEN 'Classe A (Top 80%)'
            WHEN pareto_acumulado_pct <= 95.0 THEN 'Classe B (15%)'
            ELSE 'Classe C (5%)'
        END AS classe_abc
    FROM pareto_calc
    ORDER BY posicao_ranking, produto_id;

-- 5. Receita mensal paga com crescimento MoM (LAG) e media movel de 3 meses
CREATE VIEW gold_receita_mensal AS
    WITH vendas_mensais AS (
        SELECT
            substr(p.data_pedido, 1, 7) AS mes,
            COUNT(DISTINCT p.id) AS total_pedidos,
            ROUND(SUM(ip.quantidade * ip.preco_unit), 2) AS receita_liquida
        FROM pedidos p
        JOIN itens_pedido ip ON ip.pedido_id = p.id
        WHERE p.status = 'pago'
        GROUP BY substr(p.data_pedido, 1, 7)
    ),
    com_lag AS (
        SELECT
            mes,
            total_pedidos,
            receita_liquida,
```

```
            LAG(receita_liquida) OVER (ORDER BY mes) AS receita_mes_anterior,
            ROUND(
                AVG(receita_liquida) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW),
                2
            ) AS media_movel_3m
        FROM vendas_mensais
    )
    SELECT
        mes,
        total_pedidos,
        receita_liquida,
        receita_mes_anterior,
        ROUND(
            CASE
                WHEN receita_mes_anterior IS NULL OR receita_mes_anterior = 0 THEN NULL
                ELSE 100.0 * (receita_liquida - receita_mes_anterior) / receita_mes_anterior
            END,
            1
        ) AS variacao_mom_pct,
        media_movel_3m
    FROM com_lag
    ORDER BY mes;

-- 6. Segmentacao RFM (Recencia, Frequencia, Valor Monetario sobre pedidos pagos)
CREATE VIEW gold_rfm_clientes AS
    WITH ref_date AS (
        SELECT MAX(data_pedido) AS data_max FROM pedidos WHERE status = 'pago'
    ),
    base_rfm AS (
        SELECT
            c.id AS cliente_id,
            c.nome AS cliente,
            c.cidade,
            CAST(
                julianday((SELECT data_max FROM ref_date)) - julianday(MAX(p.data_pedido))
                AS INTEGER
            ) AS recencia_dias,
            COUNT(DISTINCT p.id) AS frequencia_pedidos,
            ROUND(SUM(ip.quantidade * ip.preco_unit), 2) AS valor_monetario
        FROM clientes c
        JOIN pedidos p ON p.cliente_id = c.id
        JOIN itens_pedido ip ON ip.pedido_id = p.id
        WHERE p.status = 'pago'
        GROUP BY c.id, c.nome, c.cidade
    ),
    quartis AS (
        SELECT
            cliente_id,
            cliente,
            cidade,
            recencia_dias,
            frequencia_pedidos,
            valor_monetario,
            NTILE(4) OVER (ORDER BY recencia_dias DESC) AS r_score,
            NTILE(4) OVER (ORDER BY frequencia_pedidos ASC, valor_monetario ASC) AS f_score,
            NTILE(4) OVER (ORDER BY valor_monetario ASC) AS m_score
        FROM base_rfm
    )
    SELECT
        cliente_id,
        cliente,
        cidade,
        recencia_dias,
        frequencia_pedidos,
        valor_monetario,
        r_score,
        f_score,
        m_score,
        CASE
            WHEN r_score >= 3 AND f_score >= 3 AND m_score >= 3 THEN 'Campeoes'
            WHEN r_score >= 2 AND f_score >= 3 THEN 'Leais'
```

```
            WHEN r_score <= 2 AND (f_score >= 3 OR m_score >= 3) THEN 'Em risco'
            ELSE 'Inativos'
        END AS segmento_rfm
    FROM quartis
    ORDER BY valor_monetario DESC;

-- 7. Retencao por coorte (mes do primeiro pedido pago)
CREATE VIEW gold_coorte_retencao AS
    WITH primeira_compra AS (
        SELECT
            cliente_id,
             MIN(substr(data_pedido, 1, 7)) AS coorte_mes
        FROM pedidos
        WHERE status = 'pago'
        GROUP BY cliente_id
    ),
    atividade_mensal AS (
        SELECT DISTINCT
            p.cliente_id,
            pc.coorte_mes,
            substr(p.data_pedido, 1, 7) AS mes_pedido,
            (CAST(substr(p.data_pedido, 1, 4) AS INTEGER) - CAST(substr(pc.coorte_mes, 1, 4) AS INTEGER)) * 12
            + (CAST(substr(p.data_pedido, 6, 2) AS INTEGER) - CAST(substr(pc.coorte_mes, 6, 2) AS INTEGER)) AS
indice_mes
        FROM pedidos p
        JOIN primeira_compra pc ON pc.cliente_id = p.cliente_id
        WHERE p.status = 'pago'
    ),
    tamanho_coortes AS (
        SELECT
            coorte_mes,
            COUNT(DISTINCT cliente_id) AS clientes_iniciais
        FROM primeira_compra
        GROUP BY coorte_mes
    )
    SELECT
        a.coorte_mes,
        a.mes_pedido,
        a.indice_mes,
        COUNT(DISTINCT a.cliente_id) AS clientes_ativos,
        t.clientes_iniciais,
        ROUND(100.0 * COUNT(DISTINCT a.cliente_id) / t.clientes_iniciais, 1) AS taxa_retencao_pct
    FROM atividade_mensal a
    JOIN tamanho_coortes t ON t.coorte_mes = a.coorte_mes
    GROUP BY a.coorte_mes, a.mes_pedido, a.indice_mes, t.clientes_iniciais
    ORDER BY a.coorte_mes, a.indice_mes;

-- 8. Taxa global de recompra (sobre clientes pagantes)
CREATE VIEW gold_taxa_recompra AS
    WITH pedidos_por_cliente AS (
        SELECT
            cliente_id,
            COUNT(DISTINCT id) AS qtd_pedidos
        FROM pedidos
        WHERE status = 'pago'
        GROUP BY cliente_id
    )
    SELECT
        COUNT(cliente_id) AS clientes_compradores,
        SUM(CASE WHEN qtd_pedidos >= 2 THEN 1 ELSE 0 END) AS clientes_recorrentes,
        ROUND(
            100.0 * SUM(CASE WHEN qtd_pedidos >= 2 THEN 1 ELSE 0 END) / NULLIF(COUNT(cliente_id), 0),
            1
        ) AS taxa_recompra_pct,
        ROUND(AVG(qtd_pedidos), 2) AS media_pedidos_por_cliente
    FROM pedidos_por_cliente;

-- 9. Cancelamento por categoria e por mes
CREATE VIEW gold_cancelamento_categoria_mes AS
    SELECT
        substr(p.data_pedido, 1, 7) AS mes,
```

```
        pr.categoria,
        COUNT(DISTINCT p.id) AS total_pedidos,
        COUNT(DISTINCT CASE WHEN p.status = 'cancelado' THEN p.id END) AS pedidos_cancelados,
        ROUND(
            100.0 * COUNT(DISTINCT CASE WHEN p.status = 'cancelado' THEN p.id END)
            / NULLIF(COUNT(DISTINCT p.id), 0),
            1
        ) AS taxa_cancelamento_pct,
        ROUND(
            SUM(CASE WHEN p.status = 'cancelado' THEN ip.quantidade * ip.preco_unit ELSE 0 END),
            2
        ) AS receita_cancelada
    FROM pedidos p
    JOIN itens_pedido ip ON ip.pedido_id = p.id
    JOIN produtos pr ON pr.id = ip.produto_id
    GROUP BY substr(p.data_pedido, 1, 7), pr.categoria
    ORDER BY mes DESC, taxa_cancelamento_pct DESC;

-- 10. Receita paga e cancelamento diario (base para Carta de Controle I-MR Lean Six Sigma)
CREATE VIEW gold_receita_diaria AS
    WITH pedidos_dia AS (
        SELECT
            p.data_pedido,
            p.id AS pedido_id,
            p.status,
            SUM(ip.quantidade * ip.preco_unit) AS valor_pedido
        FROM pedidos p
        JOIN itens_pedido ip ON ip.pedido_id = p.id
        GROUP BY p.data_pedido, p.id, p.status
    )
    SELECT
        data_pedido,
        COUNT(pedido_id) AS total_pedidos,
        SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) AS pedidos_cancelados,
        ROUND(
            100.0 * SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) / NULLIF(COUNT(pedido_id), 0),
            2
        ) AS taxa_cancelamento_pct,
        ROUND(
            SUM(CASE WHEN status = 'pago' THEN valor_pedido ELSE 0 END),
            2
        ) AS receita_liquida_dia
    FROM pedidos_dia
    GROUP BY data_pedido
    ORDER BY data_pedido;

-- 11. Resumo de qualidade dos dados rejeitados na camada Silver
CREATE VIEW gold_qualidade_resumo AS
    SELECT
        tabela_origem,
        motivo,
        COUNT(*) AS total_rejeitados
    FROM silver_rejeitados
    GROUP BY tabela_origem, motivo
    ORDER BY total_rejeitados DESC, tabela_origem;
```

Arquivo: `src/config.py`

```
"""
Configuracao centralizada de caminhos, sementes e janelas de negocio do pipeline.
"""

import os

DIR_RAIZ = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DIR_SQL = os.path.join(DIR_RAIZ, "sql")
CAMINHO_BANCO = os.path.join(DIR_RAIZ, "loja_digital.db")
PASTA_RAW_PADRAO = os.path.join(DIR_RAIZ, "data", "raw")
PASTA_EXPORTS_PADRAO = os.path.join(DIR_RAIZ, "exports")
```

```
SEMENTE_PADRAO = 42
NUM_CLIENTES_PADRAO = 200
NUM_PEDIDOS_PADRAO = 5000

DATA_MIN_NEGOCIO = "2020-01-01"
DATA_MAX_NEGOCIO = "2030-12-31"
```

Arquivo: `src/main.py`

```
"""
Ponto de entrada da aplicacao.
Suporta argumentos de linha de comando (--pipeline, --demo) e menu interativo.
"""

import argparse
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import relatorios
from database import CAMINHO_BANCO, conectar, criar_banco
from pipeline.executar import run_pipeline
from pipeline.gold import construir_gold

OPCOES = {
    "1": ("Listar todos os produtos", relatorios.relatorio_produtos),
    "2": ("Filtrar produtos por categoria", relatorios.relatorio_categoria),
    "3": ("Listar clientes cadastrados", relatorios.relatorio_clientes),
    "4": ("Pedidos detalhados (View SQL)", relatorios.relatorio_pedidos),
    "5": ("Ranking de produtos mais vendidos", relatorios.relatorio_ranking),
    "6": ("Faturamento por categoria", relatorios.relatorio_faturamento),
    "7": ("Clientes por volume de compras", relatorios.relatorio_clientes_vip),
    "8": ("Clientes acima de meta (HAVING)", relatorios.relatorio_clientes_meta),
    "9": ("Produtos acima da media (Subquery)", relatorios.relatorio_produtos_acima_media),
    "10": ("Desempenho dos produtos (CTE + CASE WHEN)", relatorios.relatorio_desempenho_cte),
    "11": ("KPIs executivos e Curva de Pareto (Window Function)", relatorios.relatorio_kpis_pareto),
    "12": ("Cadastrar novo produto", relatorios.cadastrar_produto),
    "13": ("Gerar Dashboard Executivo (Matplotlib)", relatorios.gerar_grafico_faturamento),
    "14": ("Automatizar exportacao analitica (CSV)", relatorios.exportar_relatorios_csv),
    "15": ("Executar pipeline Medalha (Bronze -> Silver -> Gold)", relatorios.relatorio_executar_pipeline),
    "16": ("Relatorio de qualidade e quarentena (Silver)", relatorios.relatorio_qualidade),
    "17": ("Receita mensal, MoM e media movel 3M (Gold)", relatorios.relatorio_receita_mensal),
    "18": ("Segmentacao RFM de clientes (Gold)", relatorios.relatorio_rfm),
    "19": ("Taxa de recompra e retencao por coorte (Gold)", relatorios.relatorio_coorte_recompra),
    "20": ("Cancelamentos e Carta de Controle 3-Sigma (Gold)", relatorios.relatorio_carta_controle),
}


def exibir_menu() -> None:
    print("\n" + "=" * 62)
    print("  LOJA DIGITAL - PAINEL DE RELATORIOS E ARQUITETURA MEDALHA")
    print("=" * 62)
    for chave, (descricao, _) in OPCOES.items():
        print(f"  [{chave:>2}] {descricao}")
    print("  [ 0] Sair")
    print("-" * 62)


def _parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Loja Digital - Pipeline Medalha (Bronze, Silver, Gold) e Analytics"
    )
    parser.add_argument(
        "--pipeline",
        action="store_true",
        help="Gera CSVs sinteticos em data/raw/ e executa a carga completa (Faker -> Bronze -> Silver -> Gold).",
    )
```

```
    parser.add_argument(
        "--reprocessar",
        action="store_true",
        help="Reprocessa os CSVs ja existentes em data/raw/ (Bronze -> Silver -> Gold) sem regerar dados
sinteticos.",
    )
    parser.add_argument(
        "--demo",
        action="store_true",
        help="Inicializa o banco com o seed.sql pequeno de demonstracao antes de abrir o menu.",
    )
    return parser.parse_args(argv)


def _banco_possui_pipeline() -> bool:
    """Verifica se o arquivo SQLite ja existe e contem carga do pipeline Medalha."""
    if not os.path.exists(CAMINHO_BANCO):
        return False
    try:
        conn = conectar()
        total = conn.execute("SELECT COUNT(*) FROM pedidos").fetchone()[0]
        conn.close()
        return int(total) > 50
    except Exception:  # noqa: BLE001
        return False


def main(argv: list[str] | None = None) -> None:
    args = _parse_args(argv)

    if args.pipeline or args.reprocessar:
        if not os.path.exists(CAMINHO_BANCO):
            criar_banco()
        conn = conectar()
        try:
            run_pipeline(conn=conn, regerar_csv=not args.reprocessar)
            relatorios.gerar_grafico_faturamento(conn)
        finally:
            conn.close()
        return

    if args.demo or not _banco_possui_pipeline():
        criar_banco()
    else:
        conn_init = conectar()
        construir_gold(conn_init)
        conn_init.close()

    try:
        conn = conectar()
    except Exception:  # noqa: BLE001
        print("[ERRO] Nao foi possivel conectar ao banco. Encerrando.")
        sys.exit(1)

    try:
        while True:
            exibir_menu()

            try:
                escolha = input("  Escolha uma opcao: ").strip()
            except (EOFError, KeyboardInterrupt):
                print("\n\n  Encerrando sistema.\n")
                break

            if escolha == "0":
                print("\n  Encerrando sistema.\n")
                break

            if escolha not in OPCOES:
                print(f"\n  Opcao '{escolha}' invalida. Tente novamente.")
                continue
```

```
            _, funcao = OPCOES[escolha]

            try:
                funcao(conn)
            except Exception as erro:  # noqa: BLE001
                print(f"\n  [ERRO] Falha ao gerar relatorio: {erro}")

    finally:
        conn.close()


if __name__ == "__main__":
    main()
```

Arquivo: `src/database.py`

```
"""
Modulo responsavel pela conexao com o SQLite e criacao do banco de dados.
"""

import os
import sqlite3

from config import CAMINHO_BANCO, DIR_SQL

_DIR_SQL = DIR_SQL
_BRONZE = os.path.join(_DIR_SQL, "bronze.sql")
_SCHEMA = os.path.join(_DIR_SQL, "schema.sql")
_SEED = os.path.join(_DIR_SQL, "seed.sql")
_GOLD = os.path.join(_DIR_SQL, "gold.sql")


def conectar(caminho_banco: str | None = None) -> sqlite3.Connection:
    """Abre e retorna uma conexao com o banco SQLite com chaves estrangeiras ativas."""
    alvo = caminho_banco or CAMINHO_BANCO
    try:
        conn = sqlite3.connect(alvo)
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        return conn
    except sqlite3.Error as erro:
        print(f"[ERRO] Nao foi possivel conectar ao banco: {erro}")
        raise


def _executar_script(conn: sqlite3.Connection, caminho_sql: str) -> None:
    """Le e executa um arquivo .sql no banco."""
    try:
        with open(caminho_sql, "r", encoding="utf-8") as arquivo:
            script = arquivo.read()
        conn.executescript(script)
    except FileNotFoundError:
        print(f"[ERRO] Arquivo SQL nao encontrado: {caminho_sql}")
        raise
    except sqlite3.Error as erro:
        print(f"[ERRO] Falha ao executar script SQL: {erro}")
        raise


def criar_banco(caminho_banco: str | None = None) -> None:
    """Recria o banco de demonstracao executando bronze.sql, schema.sql, seed.sql e gold.sql."""
    alvo = caminho_banco or CAMINHO_BANCO
    if alvo != ":memory:" and os.path.exists(alvo):
        os.remove(alvo)

    try:
        conn = conectar(alvo)
        print("Inicializando banco de dados...")
```

```
        if os.path.exists(_BRONZE):
            _executar_script(conn, _BRONZE)
        _executar_script(conn, _SCHEMA)
        _executar_script(conn, _SEED)
        if os.path.exists(_GOLD):
            _executar_script(conn, _GOLD)
        conn.close()
        print("Banco de dados pronto!\n")
    except Exception as erro:
        print(f"[ERRO] Falha ao criar o banco: {erro}")
        raise
```

Arquivo: `src/queries.py`

```
"""
Consultas SQL utilizadas nos relatorios do sistema.
"""

import sqlite3
from typing import Any


def listar_produtos(conn: sqlite3.Connection) -> list[Any]:
    """Retorna todos os produtos cadastrados."""
    cursor = conn.execute(
        """
        SELECT id, nome, categoria, preco, estoque
        FROM produtos
        ORDER BY categoria, nome
        """
    )
    return cursor.fetchall()


def filtrar_por_categoria(conn: sqlite3.Connection, categoria: str) -> list[Any]:
    """Filtra produtos por categoria (case-insensitive)."""
    cursor = conn.execute(
        """
        SELECT id, nome, preco, estoque
        FROM produtos
        WHERE LOWER(categoria) = LOWER(?)
        ORDER BY preco
        """,
        (categoria,),
    )
    return cursor.fetchall()


def listar_clientes(conn: sqlite3.Connection) -> list[Any]:
    """Retorna todos os clientes ordenados por nome."""
    cursor = conn.execute(
        """
        SELECT id, nome, email, cidade, criado_em
        FROM clientes
        ORDER BY nome
        """
    )
    return cursor.fetchall()


def resumo_por_cliente(conn: sqlite3.Connection) -> list[Any]:
    """Total gasto e quantidade de pedidos por cliente (ignora cancelados)."""
    cursor = conn.execute(
        """
        SELECT
            c.nome AS cliente,
            c.cidade,
            COUNT(DISTINCT p.id) AS total_pedidos,
            ROUND(SUM(ip.quantidade * ip.preco_unit), 2) AS total_gasto
```

```
        FROM clientes c
        JOIN pedidos p ON p.cliente_id = c.id
        JOIN itens_pedido ip ON ip.pedido_id = p.id
        WHERE p.status != 'cancelado'
        GROUP BY c.id, c.nome, c.cidade
        ORDER BY total_gasto DESC
        """
    )
    return cursor.fetchall()


def produtos_mais_vendidos(conn: sqlite3.Connection) -> list[Any]:
    """Ranking de produtos por volume de vendas e receita."""
    cursor = conn.execute(
        """
        SELECT
            pr.nome AS produto,
            pr.categoria,
            SUM(ip.quantidade) AS unidades_vendidas,
            ROUND(SUM(ip.quantidade * ip.preco_unit), 2) AS receita_total
        FROM produtos pr
        JOIN itens_pedido ip ON ip.produto_id = pr.id
        JOIN pedidos p ON p.id = ip.pedido_id
        WHERE p.status != 'cancelado'
        GROUP BY pr.id, pr.nome, pr.categoria
        ORDER BY unidades_vendidas DESC
        """
    )
    return cursor.fetchall()


def pedidos_detalhados(conn: sqlite3.Connection) -> list[Any]:
    """Consulta a view resumo_pedidos com os dados consolidados de cada pedido."""
    cursor = conn.execute(
        """
        SELECT *
        FROM resumo_pedidos
        ORDER BY data_pedido DESC
        """
    )
    return cursor.fetchall()


def faturamento_por_categoria(conn: sqlite3.Connection) -> list[Any]:
    """Agrupa vendas, ticket medio e receita por categoria de produto."""
    cursor = conn.execute(
        """
        SELECT
            pr.categoria,
            COUNT(DISTINCT pr.id) AS produtos_distintos,
            SUM(ip.quantidade) AS unidades_vendidas,
            ROUND(AVG(ip.preco_unit), 2) AS ticket_medio,
            ROUND(SUM(ip.quantidade * ip.preco_unit), 2) AS receita_total
        FROM produtos pr
        JOIN itens_pedido ip ON ip.produto_id = pr.id
        JOIN pedidos p ON p.id = ip.pedido_id
        WHERE p.status != 'cancelado'
        GROUP BY pr.categoria
        ORDER BY receita_total DESC
        """
    )
    return cursor.fetchall()


def clientes_acima_de_meta(conn: sqlite3.Connection, meta: float = 200.0) -> list[Any]:
    """Filtra clientes cujo gasto total supera a meta informada (usando HAVING)."""
    cursor = conn.execute(
        """
        SELECT
            c.nome AS cliente,
            c.email,
```

```
            c.cidade,
            COUNT(DISTINCT p.id) AS total_pedidos,
            ROUND(SUM(ip.quantidade * ip.preco_unit), 2) AS total_gasto
        FROM clientes c
        JOIN pedidos p ON p.cliente_id = c.id
        JOIN itens_pedido ip ON ip.pedido_id = p.id
        WHERE p.status != 'cancelado'
        GROUP BY c.id, c.nome, c.email, c.cidade
        HAVING SUM(ip.quantidade * ip.preco_unit) > ?
        ORDER BY total_gasto DESC
        """,
        (meta,),
    )
    return cursor.fetchall()


def produtos_acima_da_media(conn: sqlite3.Connection) -> list[Any]:
    """Lista produtos com preco acima da media geral usando subconsulta."""
    cursor = conn.execute(
        """
        SELECT
            id,
            nome,
            categoria,
            preco,
            ROUND(preco - (SELECT AVG(preco) FROM produtos), 2) AS diferenca_da_media
        FROM produtos
        WHERE preco > (SELECT AVG(preco) FROM produtos)
        ORDER BY preco DESC
        """
    )
    return cursor.fetchall()


def desempenho_produtos_cte(conn: sqlite3.Connection) -> list[Any]:
    """Classifica os produtos em faixas de receita usando CTE e CASE WHEN."""
    cursor = conn.execute(
        """
        WITH vendas_por_produto AS (
            SELECT
                pr.id,
                pr.nome AS produto,
                pr.categoria,
                pr.preco AS preco_unitario,
                COALESCE(SUM(ip.quantidade), 0) AS unidades_vendidas,
                ROUND(COALESCE(SUM(ip.quantidade * ip.preco_unit), 0), 2) AS receita_total
            FROM produtos pr
            LEFT JOIN itens_pedido ip ON ip.produto_id = pr.id
            LEFT JOIN pedidos p ON p.id = ip.pedido_id
                               AND p.status != 'cancelado'
            GROUP BY pr.id, pr.nome, pr.categoria, pr.preco
        )
        SELECT
            produto,
            categoria,
            preco_unitario,
            unidades_vendidas,
            receita_total,
            CASE
                WHEN receita_total > 400 THEN 'Alto'
                WHEN receita_total >= 100 THEN 'Medio'
                ELSE 'Baixo'
            END AS desempenho
        FROM vendas_por_produto
        ORDER BY receita_total DESC
        """
    )
    return cursor.fetchall()


def inserir_produto(
```

```
    conn: sqlite3.Connection, nome: str, categoria: str, preco: float, estoque: int
) -> int:
    """Insere um novo produto no banco e retorna o ID gerado."""
    cursor = conn.execute(
        """
        INSERT INTO produtos (nome, categoria, preco, estoque)
        VALUES (?, ?, ?, ?)
        """,
        (nome, categoria, preco, estoque),
    )
    conn.commit()
    return int(cursor.lastrowid)


def calcular_kpis_executivos(conn: sqlite3.Connection) -> Any:
    """Calcula os principais KPIs executivos de vendas, ticket medio, LTV e conversao."""
    cursor = conn.execute(
        """
        WITH totais_pedidos AS (
            SELECT
                p.id AS pedido_id,
                p.cliente_id,
                p.status,
                SUM(ip.quantidade * ip.preco_unit) AS valor_pedido
            FROM pedidos p
            JOIN itens_pedido ip ON ip.pedido_id = p.id
            GROUP BY p.id, p.cliente_id, p.status
        )
        SELECT
            COUNT(pedido_id) AS total_pedidos,
            SUM(CASE WHEN status = 'pago' THEN 1 ELSE 0 END) AS pedidos_pagos,
            ROUND(SUM(CASE WHEN status = 'pago' THEN valor_pedido ELSE 0 END), 2) AS receita_paga,
            ROUND(SUM(CASE WHEN status = 'pendente' THEN valor_pedido ELSE 0 END), 2) AS receita_pendente,
            ROUND(AVG(CASE WHEN status = 'pago' THEN valor_pedido END), 2) AS ticket_medio_pedido,
            ROUND(
                SUM(CASE WHEN status = 'pago' THEN valor_pedido ELSE 0 END)
                / NULLIF(COUNT(DISTINCT CASE WHEN status = 'pago' THEN cliente_id END), 0),
                2
            ) AS ltv_medio_cliente,
            ROUND(
                100.0 * SUM(CASE WHEN status = 'pago' THEN 1 ELSE 0 END) / COUNT(pedido_id),
                1
            ) AS taxa_conversao_pct,
            ROUND(
                100.0 * SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) / COUNT(pedido_id),
                1
            ) AS taxa_cancelamento_pct
        FROM totais_pedidos
        """
    )
    return cursor.fetchone()


def ranking_window_function(conn: sqlite3.Connection) -> list[Any]:
    """Calcula ranking e participacao percentual (share) usando Window Functions (RANK e SUM OVER)."""
    cursor = conn.execute(
        """
        WITH receita_produtos AS (
            SELECT
                pr.nome AS produto,
                pr.categoria,
                COALESCE(SUM(ip.quantidade), 0) AS unidades,
                ROUND(COALESCE(SUM(ip.quantidade * ip.preco_unit), 0), 2) AS receita
            FROM produtos pr
            LEFT JOIN itens_pedido ip ON ip.produto_id = pr.id
            LEFT JOIN pedidos p ON p.id = ip.pedido_id AND p.status != 'cancelado'
            GROUP BY pr.id, pr.nome, pr.categoria
        )
        SELECT
            RANK() OVER (ORDER BY receita DESC) AS posicao_ranking,
            produto,
```

```
            categoria,
            unidades,
            receita,
            ROUND(100.0 * receita / SUM(receita) OVER (), 1) AS share_receita_pct,
            ROUND(
                100.0 * SUM(receita) OVER (ORDER BY receita DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW)
                / SUM(receita) OVER (),
                1
            ) AS pareto_acumulado_pct
        FROM receita_produtos
        ORDER BY posicao_ranking
        """
    )
    return cursor.fetchall()


def receita_mensal_mom(conn: sqlite3.Connection) -> list[Any]:
    """Consulta a view gold_receita_mensal com crescimento MoM (LAG) e media movel 3M."""
    cursor = conn.execute("SELECT * FROM gold_receita_mensal ORDER BY mes")
    return cursor.fetchall()


def segmentacao_rfm(conn: sqlite3.Connection) -> list[Any]:
    """Consulta a view gold_rfm_clientes com scores R, F, M e segmento atribuido."""
    cursor = conn.execute("SELECT * FROM gold_rfm_clientes ORDER BY valor_monetario DESC")
    return cursor.fetchall()


def resumo_segmentos_rfm(conn: sqlite3.Connection) -> list[Any]:
    """Agrega contagem de clientes, frequencia media e receita total por segmento RFM."""
    cursor = conn.execute(
        """
        SELECT
            segmento_rfm,
            COUNT(cliente_id) AS total_clientes,
            ROUND(AVG(recencia_dias), 1) AS recencia_media_dias,
            ROUND(AVG(frequencia_pedidos), 1) AS frequencia_media,
            ROUND(SUM(valor_monetario), 2) AS receita_segmento,
            ROUND(100.0 * SUM(valor_monetario) / NULLIF(SUM(SUM(valor_monetario)) OVER (), 0), 1) AS
share_receita_pct
        FROM gold_rfm_clientes
        GROUP BY segmento_rfm
        ORDER BY receita_segmento DESC
        """
    )
    return cursor.fetchall()


def retencao_coorte(conn: sqlite3.Connection) -> list[Any]:
    """Consulta a matriz de retencao por coorte mensal na view gold_coorte_retencao."""
    cursor = conn.execute(
        "SELECT * FROM gold_coorte_retencao ORDER BY coorte_mes, indice_mes"
    )
    return cursor.fetchall()


def taxa_recompra_global(conn: sqlite3.Connection) -> Any:
    """Consulta a taxa de recompra global na view gold_taxa_recompra."""
    cursor = conn.execute("SELECT * FROM gold_taxa_recompra")
    return cursor.fetchone()


def cancelamento_categoria_mes(conn: sqlite3.Connection) -> list[Any]:
    """Consulta a taxa de cancelamento e receita perdida por categoria e mes."""
    cursor = conn.execute(
        "SELECT * FROM gold_cancelamento_categoria_mes ORDER BY mes DESC, taxa_cancelamento_pct DESC"
    )
    return cursor.fetchall()
```

```
def resumo_qualidade_dados(conn: sqlite3.Connection) -> list[Any]:
    """Consulta a contagem de registros rejeitados na camada Silver por tabela e motivo."""
    cursor = conn.execute(
        "SELECT * FROM gold_qualidade_resumo ORDER BY total_rejeitados DESC, tabela_origem"
    )
    return cursor.fetchall()


def historico_execucoes_pipeline(conn: sqlite3.Connection) -> list[Any]:
    """Retorna o historico auditavel de execucoes do pipeline registrado em pipeline_execucoes."""
    cursor = conn.execute(
        "SELECT * FROM pipeline_execucoes ORDER BY run_id DESC"
    )
    return cursor.fetchall()
```

Arquivo: `src/pipeline/executar.py`

```
"""
Orquestrador principal do pipeline em Arquitetura Medalha (Bronze -> Silver -> Gold).
Garante execucao idempotente, separacao entre geracao/reprocessamento e auditoria em pipeline_execucoes.
"""

import logging
import os
import sqlite3
from datetime import UTC, datetime
from typing import Any

from config import (
    NUM_CLIENTES_PADRAO,
    NUM_PEDIDOS_PADRAO,
    PASTA_EXPORTS_PADRAO,
    PASTA_RAW_PADRAO,
    SEMENTE_PADRAO,
)
from database import conectar
from pipeline.bronze import carregar_bronze
from pipeline.gerar_dados import gerar_arquivos_raw
from pipeline.gold import construir_gold, exportar_camada_gold
from pipeline.qualidade import gerar_relatorio_qualidade
from pipeline.silver import processar_silver

logger = logging.getLogger(__name__)


def _configurar_logging() -> None:
    if not logging.getLogger().handlers:
        logging.basicConfig(
            level=logging.INFO,
            format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
            datefmt="%Y-%m-%d %H:%M:%S",
        )


def _registrar_execucao_pipeline(
    conn: sqlite3.Connection,
    inicio_iso: str,
    fim_iso: str,
    status: str,
    modo_geracao: str,
    total_bronze: int,
    total_silver: int,
    total_rejeitados: int,
) -> dict[str, Any]:
    """Registra metadados de rastreabilidade da execucao na tabela pipeline_execucoes."""
    conn.execute(
        """
        CREATE TABLE IF NOT EXISTS pipeline_execucoes (
            run_id              INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            inicio_execucao     TEXT    NOT NULL,
            fim_execucao        TEXT    NOT NULL,
            status              TEXT    NOT NULL,
            modo_geracao        TEXT    NOT NULL DEFAULT 'regerado',
            registros_bronze    INTEGER NOT NULL DEFAULT 0,
            registros_silver    INTEGER NOT NULL DEFAULT 0,
            registros_rejeitados INTEGER NOT NULL DEFAULT 0
        )
        """
    )
    cursor = conn.execute(
        """
        INSERT INTO pipeline_execucoes (
            inicio_execucao,
            fim_execucao,
            status,
            modo_geracao,
            registros_bronze,
            registros_silver,
            registros_rejeitados
        )
        VALUES (?, ?, ?, ?, ?, ?, ?)
        """,
        (
            inicio_iso,
            fim_iso,
            status,
            modo_geracao,
            total_bronze,
            total_silver,
            total_rejeitados,
        ),
    )
    conn.commit()
    run_id = int(cursor.lastrowid)
    logger.info(
        "Execucao registrada em pipeline_execucoes (run_id=%d, status=%s, modo=%s)",
        run_id,
        status,
        modo_geracao,
    )
    return {
        "run_id": run_id,
        "inicio_execucao": inicio_iso,
        "fim_execucao": fim_iso,
        "status": status,
        "modo_geracao": modo_geracao,
        "registros_bronze": total_bronze,
        "registros_silver": total_silver,
        "registros_rejeitados": total_rejeitados,
    }


def run_pipeline(
    conn: sqlite3.Connection | None = None,
    pasta_raw: str = PASTA_RAW_PADRAO,
    pasta_exports: str = PASTA_EXPORTS_PADRAO,
    semente: int = SEMENTE_PADRAO,
    num_clientes: int = NUM_CLIENTES_PADRAO,
    num_pedidos: int = NUM_PEDIDOS_PADRAO,
    regerar_csv: bool = True,
) -> dict[str, Any]:
    """
    Executa o pipeline completo:
      1. Gera CSVs brutos em data/raw/ com Faker (se regerar_csv=True ou ausentes)
      2. Carrega camada Bronze (imutavel com _ingested_at e _source_file)
      3. Processa camada Silver (tipagem, FKs e quarentena silver_rejeitados)
      4. Gera relatorio de qualidade de dados
      5. Constroi views analiticas da camada Gold e exporta CSVs em exports/
      6. Persiste metadados da execucao em pipeline_execucoes
    """
```

```
    _configurar_logging()
    inicio_iso = datetime.now(UTC).strftime("%Y-%m-%dT%H:%M:%SZ")
    fechar_ao_final = False
    if conn is None:
        conn = conectar()
        fechar_ao_final = True

    try:
        arquivos_esperados = [
            os.path.join(pasta_raw, arq)
            for arq in ("clientes.csv", "produtos.csv", "pedidos.csv", "itens_pedido.csv")
        ]
        if regerar_csv or not all(os.path.exists(p) for p in arquivos_esperados):
            gerados = gerar_arquivos_raw(
                pasta_destino=pasta_raw,
                semente=semente,
                num_clientes=num_clientes,
                num_pedidos=num_pedidos,
            )
            modo_geracao = "regerado"
        else:
            gerados = {}
            modo_geracao = "reprocessado"

        contagens_bronze = carregar_bronze(conn, pasta_raw=pasta_raw)
        metricas_silver = processar_silver(conn)
        resumo_qualidade = gerar_relatorio_qualidade(conn, pasta_exports=pasta_exports)
        construir_gold(conn)
        exportados_gold = exportar_camada_gold(conn, pasta_exports=pasta_exports)

        fim_iso = datetime.now(UTC).strftime("%Y-%m-%dT%H:%M:%SZ")
        total_bronze = sum(contagens_bronze.values())
        total_silver = sum(m["validas"] for m in metricas_silver.values())
        total_rejeitados = sum(m["rejeitadas"] for m in metricas_silver.values())

        execucao_info = _registrar_execucao_pipeline(
            conn=conn,
            inicio_iso=inicio_iso,
            fim_iso=fim_iso,
            status="sucesso",
            modo_geracao=modo_geracao,
            total_bronze=total_bronze,
            total_silver=total_silver,
            total_rejeitados=total_rejeitados,
        )

        return {
            "raw": gerados,
            "bronze": contagens_bronze,
            "silver": metricas_silver,
            "qualidade": resumo_qualidade,
            "exports": exportados_gold,
            "execucao": execucao_info,
        }
    finally:
        if fechar_ao_final:
            conn.close()
```

Arquivo: `src/pipeline/qualidade.py`

```
"""
Regras de validacao de qualidade de dados para a transicao Bronze -> Silver.
Cada regra possui funcao isolada para facilitar testes unitarios e auditoria.
"""

import csv
import logging
import math
import os
```

```
import re
import sqlite3
from datetime import date
from typing import Any

from config import DATA_MAX_NEGOCIO, DATA_MIN_NEGOCIO, PASTA_EXPORTS_PADRAO

logger = logging.getLogger(__name__)

STATUS_PEDIDO_VALIDOS = {"pago", "pendente", "cancelado"}
_REGEX_EMAIL = re.compile(r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")
_DATA_MIN = date.fromisoformat(DATA_MIN_NEGOCIO)
_DATA_MAX = date.fromisoformat(DATA_MAX_NEGOCIO)


def validar_obrigatorios(
    registro: dict[str, Any], campos: list[str]
) -> tuple[bool, str | None]:
    """Verifica se todos os campos obrigatorios estao preenchidos e nao nulos."""
    for campo in campos:
        valor = registro.get(campo)
        if valor is None or str(valor).strip() == "":
            return False, f"nulo_obrigatorio:{campo}"
    return True, None


def validar_chave_duplicada(
    valor_id: Any, ids_vistos: set[int]
) -> tuple[bool, str | None]:
    """
    Verifica se o ID e um inteiro positivo em formato canonico e se ainda nao apareceu no lote.
    Nota: a funcao apenas valida; o chamador adiciona o ID ao conjunto apos aprovar todas as regras.
    """
    texto = str(valor_id or "").strip()
    if not texto.isdigit():
        return False, "chave_invalida"
    id_int = int(texto)
    if id_int <= 0:
        return False, "chave_invalida"

    if id_int in ids_vistos:
        return False, "chave_duplicada"
    return True, None


def validar_numero_positivo(
    valor: Any, nome_campo: str = "valor"
) -> tuple[bool, str | None]:
    """Verifica se o valor numerico finito (preco ou quantidade) e estritamente maior que zero."""
    try:
        numero = float(str(valor).strip())
    except (ValueError, TypeError):
        return False, f"valor_nao_numerico:{nome_campo}"

    if not math.isfinite(numero):
        return False, f"valor_nao_numerico:{nome_campo}"

    if numero <= 0:
        return False, f"valor_menor_ou_igual_zero:{nome_campo}"
    return True, None


def validar_inteiro_nao_negativo(
    valor: Any, nome_campo: str = "estoque"
) -> tuple[bool, str | None]:
    """Verifica se o valor e um inteiro maior ou igual a zero."""
    try:
        numero = int(str(valor).strip())
    except (ValueError, TypeError):
        return False, f"inteiro_invalido:{nome_campo}"
```

```
    if numero < 0:
        return False, f"inteiro_negativo:{nome_campo}"
    return True, None


def validar_status_pedido(status: Any) -> tuple[bool, str | None]:
    """Verifica se o status do pedido pertence ao dominio permitido."""
    texto = str(status or "").strip().lower()
    if texto not in STATUS_PEDIDO_VALIDOS:
        return False, "status_fora_do_dominio"
    return True, None


def validar_email(
    email: Any, emails_vistos: set[str] | None = None
) -> tuple[bool, str | None]:
    """Valida formato de e-mail e unicidade opcional no lote."""
    texto = str(email or "").strip().lower()
    if not _REGEX_EMAIL.match(texto):
        return False, "email_formato_invalido"
    if emails_vistos is not None and texto in emails_vistos:
        return False, "email_duplicado"
    return True, None


def validar_data_iso(
    valor_data: Any, nome_campo: str = "data"
) -> tuple[bool, str | None]:
    """Verifica se a string representa uma data ISO valida dentro da janela temporal de negocio."""
    texto = str(valor_data or "").strip()
    try:
        data_obj = date.fromisoformat(texto)
    except ValueError:
        return False, f"data_invalida:{nome_campo}"

    if not (_DATA_MIN <= data_obj <= _DATA_MAX):
        return False, f"data_fora_janela_negocio:{nome_campo}"
    return True, None


def validar_fk_existente(
    valor_fk: Any, ids_pai_validos: set[int], nome_fk: str = "fk"
) -> tuple[bool, str | None]:
    """Verifica se a chave estrangeira aponta para um registro pai valido."""
    try:
        fk_int = int(str(valor_fk).strip())
    except (ValueError, TypeError):
        return False, f"fk_orfa:{nome_fk}"

    if fk_int not in ids_pai_validos:
        return False, f"fk_orfa:{nome_fk}"
    return True, None


def gerar_relatorio_qualidade(
    conn: sqlite3.Connection, pasta_exports: str = PASTA_EXPORTS_PADRAO
) -> list[dict[str, Any]]:
    """
    Consulta a contagem de registros em silver_rejeitados por tabela e motivo,
    registra no logging e exporta para exports/qualidade_rejeitados.csv.
    """
    os.makedirs(pasta_exports, exist_ok=True)
    cursor = conn.execute(
        """
        SELECT
            tabela_origem,
            motivo,
            COUNT(*) AS total_rejeitados
        FROM silver_rejeitados
        GROUP BY tabela_origem, motivo
        ORDER BY total_rejeitados DESC, tabela_origem, motivo
```

```
        """
    )
    linhas = [dict(row) for row in cursor.fetchall()]

    caminho_csv = os.path.join(pasta_exports, "qualidade_rejeitados.csv")
    with open(caminho_csv, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(
            f, fieldnames=["tabela_origem", "motivo", "total_rejeitados"]
        )
        writer.writeheader()
        writer.writerows(linhas)

    total_geral = sum(item["total_rejeitados"] for item in linhas)
    logger.info(
        "Relatorio de qualidade gerado em %s (%d rejeicoes totais em %d motivos)",
        caminho_csv,
        total_geral,
        len(linhas),
    )
    for item in linhas:
        logger.info(
            "  [QUARENTENA] %s | %s: %d registro(s)",
            item["tabela_origem"],
            item["motivo"],
            item["total_rejeitados"],
        )

    return linhas
```

Arquivo: `src/pipeline/silver.py`

```
"""
Camada Silver: limpeza, tipagem, deduplicacao, validacao de integridade
e envio de registros invalidos para a quarentena (silver_rejeitados).
"""

import json
import logging
import os
import sqlite3
from typing import Any

from pipeline.qualidade import (
    validar_chave_duplicada,
    validar_data_iso,
    validar_email,
    validar_fk_existente,
    validar_inteiro_nao_negativo,
    validar_numero_positivo,
    validar_obrigatorios,
    validar_status_pedido,
)

logger = logging.getLogger(__name__)

_DIR_RAIZ = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
_SQL_SCHEMA = os.path.join(_DIR_RAIZ, "sql", "schema.sql")


def _garantir_tabelas_silver(conn: sqlite3.Connection) -> None:
    """Garante que as tabelas Silver existam e limpa os dados anteriores de forma idempotente."""
    conn.execute("PRAGMA foreign_keys = OFF")
    with open(_SQL_SCHEMA, "r", encoding="utf-8") as f:
        conn.executescript(f.read())
    conn.execute("PRAGMA foreign_keys = ON")


def _registrar_rejeitado(
    rejeitados: list[tuple[str, str, str]],
```

```
    tabela_origem: str,
    motivo: str,
    registro: dict[str, Any],
) -> None:
    dados_limpos = {
        k: v for k, v in registro.items() if not k.startswith("_")
    }
    rejeitados.append(
        (tabela_origem, motivo, json.dumps(dados_limpos, ensure_ascii=False))
    )


def processar_silver(conn: sqlite3.Connection) -> dict[str, dict[str, int]]:
    """
    Transforma os dados de bronze_* para as tabelas Silver (clientes, produtos,
    pedidos, itens_pedido) respeitando a ordem de chaves estrangeiras e isolando
    anomalias em silver_rejeitados.
    """
    _garantir_tabelas_silver(conn)

    rejeitados_batch: list[tuple[str, str, str]] = []
    metricas: dict[str, dict[str, int]] = {}

    # 1. CLIENTES
    rows_cli = [
        dict(r) for r in conn.execute("SELECT * FROM bronze_clientes").fetchall()
    ]
    clientes_validos: list[tuple[int, str, str, str, str]] = []
    ids_clientes: set[int] = set()
    emails_clientes: set[str] = set()
    rej_cli = 0

    for row in rows_cli:
        ok, motivo = validar_obrigatorios(
            row, ["id", "nome", "email", "cidade", "criado_em"]
        )
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_clientes", motivo or "nulo", row)
            rej_cli += 1
            continue

        ok, motivo = validar_chave_duplicada(row["id"], ids_clientes)
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_clientes", motivo or "id_invalido", row)
            rej_cli += 1
            continue

        ok, motivo = validar_email(row["email"], emails_clientes)
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_clientes", motivo or "email_invalido", row)
            rej_cli += 1
            continue

        ok, motivo = validar_data_iso(row["criado_em"], "criado_em")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_clientes", motivo or "data_invalida", row)
            rej_cli += 1
            continue

        cid = int(str(row["id"]).strip())
        email_limpo = str(row["email"]).strip().lower()
        ids_clientes.add(cid)
        emails_clientes.add(email_limpo)
        clientes_validos.append(
            (
                cid,
                str(row["nome"]).strip(),
                email_limpo,
                str(row["cidade"]).strip(),
                str(row["criado_em"]).strip(),
            )
```

```
        )

    conn.executemany(
        "INSERT INTO clientes (id, nome, email, cidade, criado_em) VALUES (?, ?, ?, ?, ?)",
        clientes_validos,
    )
    metricas["clientes"] = {
        "lidas": len(rows_cli),
        "validas": len(clientes_validos),
        "rejeitadas": rej_cli,
    }
    logger.info(
        "bronze_clientes: %d linhas -> silver clientes: %d validas, %d rejeitadas",
        len(rows_cli),
        len(clientes_validos),
        rej_cli,
    )

    # 2. PRODUTOS
    rows_prod = [
        dict(r) for r in conn.execute("SELECT * FROM bronze_produtos").fetchall()
    ]
    produtos_validos: list[tuple[int, str, str, float, int]] = []
    ids_produtos: set[int] = set()
    rej_prod = 0

    for row in rows_prod:
        ok, motivo = validar_obrigatorios(
            row, ["id", "nome", "categoria", "preco", "estoque"]
        )
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_produtos", motivo or "nulo", row)
            rej_prod += 1
            continue

        ok, motivo = validar_chave_duplicada(row["id"], ids_produtos)
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_produtos", motivo or "id_invalido", row)
            rej_prod += 1
            continue

        ok, motivo = validar_numero_positivo(row["preco"], "preco")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_produtos", motivo or "preco_invalido", row)
            rej_prod += 1
            continue

        ok, motivo = validar_inteiro_nao_negativo(row["estoque"], "estoque")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_produtos", motivo or "estoque_invalido", row)
            rej_prod += 1
            continue

        pr_id = int(str(row["id"]).strip())
        ids_produtos.add(pr_id)
        produtos_validos.append(
            (
                pr_id,
                str(row["nome"]).strip(),
                str(row["categoria"]).strip(),
                round(float(str(row["preco"]).strip()), 2),
                int(str(row["estoque"]).strip()),
            )
        )

    conn.executemany(
        "INSERT INTO produtos (id, nome, categoria, preco, estoque) VALUES (?, ?, ?, ?, ?)",
        produtos_validos,
    )
    metricas["produtos"] = {
        "lidas": len(rows_prod),
```

```
        "validas": len(produtos_validos),
        "rejeitadas": rej_prod,
    }
    logger.info(
        "bronze_produtos: %d linhas -> silver produtos: %d validas, %d rejeitadas",
        len(rows_prod),
        len(produtos_validos),
        rej_prod,
    )

    # 3. PEDIDOS
    rows_ped = [
        dict(r) for r in conn.execute("SELECT * FROM bronze_pedidos").fetchall()
    ]
    pedidos_validos: list[tuple[int, int, str, str]] = []
    ids_pedidos: set[int] = set()
    rej_ped = 0

    for row in rows_ped:
        ok, motivo = validar_obrigatorios(
            row, ["id", "cliente_id", "data_pedido", "status"]
        )
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_pedidos", motivo or "nulo", row)
            rej_ped += 1
            continue

        ok, motivo = validar_chave_duplicada(row["id"], ids_pedidos)
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_pedidos", motivo or "id_invalido", row)
            rej_ped += 1
            continue

        ok, motivo = validar_fk_existente(row["cliente_id"], ids_clientes, "cliente_id")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_pedidos", motivo or "fk_orfa", row)
            rej_ped += 1
            continue

        ok, motivo = validar_data_iso(row["data_pedido"], "data_pedido")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_pedidos", motivo or "data_invalida", row)
            rej_ped += 1
            continue

        ok, motivo = validar_status_pedido(row["status"])
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_pedidos", motivo or "status_invalido", row)
            rej_ped += 1
            continue

        pid = int(str(row["id"]).strip())
        cid = int(str(row["cliente_id"]).strip())
        ids_pedidos.add(pid)
        pedidos_validos.append(
            (
                pid,
                cid,
                str(row["data_pedido"]).strip(),
                str(row["status"]).strip().lower(),
            )
        )

    conn.executemany(
        "INSERT INTO pedidos (id, cliente_id, data_pedido, status) VALUES (?, ?, ?, ?)",
        pedidos_validos,
    )
    metricas["pedidos"] = {
        "lidas": len(rows_ped),
        "validas": len(pedidos_validos),
        "rejeitadas": rej_ped,
```

```
    }
    logger.info(
        "bronze_pedidos: %d linhas -> silver pedidos: %d validas, %d rejeitadas",
        len(rows_ped),
        len(pedidos_validos),
        rej_ped,
    )

    # 4. ITENS DE PEDIDO
    rows_itens = [
        dict(r) for r in conn.execute("SELECT * FROM bronze_itens_pedido").fetchall()
    ]
    itens_validos: list[tuple[int, int, int, int, float]] = []
    ids_itens: set[int] = set()
    rej_itens = 0

    for row in rows_itens:
        ok, motivo = validar_obrigatorios(
            row, ["id", "pedido_id", "produto_id", "quantidade", "preco_unit"]
        )
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_itens_pedido", motivo or "nulo", row)
            rej_itens += 1
            continue

        ok, motivo = validar_chave_duplicada(row["id"], ids_itens)
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_itens_pedido", motivo or "id_invalido", row)
            rej_itens += 1
            continue

        ok, motivo = validar_fk_existente(row["pedido_id"], ids_pedidos, "pedido_id")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_itens_pedido", motivo or "fk_orfa", row)
            rej_itens += 1
            continue

        ok, motivo = validar_fk_existente(row["produto_id"], ids_produtos, "produto_id")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_itens_pedido", motivo or "fk_orfa", row)
            rej_itens += 1
            continue

        ok, motivo = validar_numero_positivo(row["quantidade"], "quantidade")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_itens_pedido", motivo or "qtd_invalida", row)
            rej_itens += 1
            continue

        ok, motivo = validar_inteiro_nao_negativo(row["quantidade"], "quantidade")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_itens_pedido", motivo or "qtd_invalida", row)
            rej_itens += 1
            continue

        ok, motivo = validar_numero_positivo(row["preco_unit"], "preco_unit")
        if not ok:
            _registrar_rejeitado(rejeitados_batch, "bronze_itens_pedido", motivo or "preco_invalido", row)
            rej_itens += 1
            continue

        iid = int(str(row["id"]).strip())
        ped_id = int(str(row["pedido_id"]).strip())
        prod_id = int(str(row["produto_id"]).strip())
        qtd = int(str(row["quantidade"]).strip())
        preco_u = round(float(str(row["preco_unit"]).strip()), 2)

        ids_itens.add(iid)
        itens_validos.append((iid, ped_id, prod_id, qtd, preco_u))

    conn.executemany(
```

```
        "INSERT INTO itens_pedido (id, pedido_id, produto_id, quantidade, preco_unit) VALUES (?, ?, ?, ?, ?)",
        itens_validos,
    )
    metricas["itens_pedido"] = {
        "lidas": len(rows_itens),
        "validas": len(itens_validos),
        "rejeitadas": rej_itens,
    }
    logger.info(
        "bronze_itens_pedido: %d linhas -> silver itens_pedido: %d validas, %d rejeitadas",
        len(rows_itens),
        len(itens_validos),
        rej_itens,
    )

    conn.executemany(
        "INSERT INTO silver_rejeitados (tabela_origem, motivo, registro_raw) VALUES (?, ?, ?)",
        rejeitados_batch,
    )
    conn.commit()
    return metricas
```

Arquivo: `src/pipeline/gold.py`

```
"""
Camada Gold: materializacao das views analiticas, calculo estatistico da
Carta de Controle Lean Six Sigma (3-Sigma) com pandas e exportacao de CSVs.
"""

import csv
import logging
import os
import sqlite3
from typing import Any

import pandas as pd

logger = logging.getLogger(__name__)

_DIR_RAIZ = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
_SQL_GOLD = os.path.join(_DIR_RAIZ, "sql", "gold.sql")
PASTA_EXPORTS_PADRAO = os.path.join(_DIR_RAIZ, "exports")


def construir_gold(conn: sqlite3.Connection) -> None:
    """Executa o script sql/gold.sql para criar/atualizar todas as views gold_*."""
    with open(_SQL_GOLD, "r", encoding="utf-8") as f:
        conn.executescript(f.read())
    conn.commit()
    logger.info("Camada Gold atualizada com sucesso (views gold_* prontas).")


def _garantir_gold(conn: sqlite3.Connection) -> None:
    """Garante que as views Gold existam sem recria-las ou logar repetidamente."""
    existe = conn.execute(
        "SELECT 1 FROM sqlite_master WHERE type = 'view' AND name = 'gold_receita_diaria'"
    ).fetchone()
    if not existe:
        construir_gold(conn)


def calcular_carta_controle_diaria(
    conn: sqlite3.Connection, coluna_metrica: str = "receita_liquida_dia"
) -> dict[str, Any]:
    """
    Calcula a Carta de Controle de Individuos e Amplitude Movel (I-MR) do Lean Six Sigma
    ajustada para sazonalidade sobre a serie diaria da camada Gold:
      - Tendencia sazonal local (T_t): media movel centrada de 15 dias
      - Residuo dessazonalizado (e_t): y_t - T_t
```

```
  - Amplitude Movel (MR_t): |e_t - e_{t-1}|
  - Sigma de curto prazo (I-MR): sigma_imr = MR_medio / 1.128 (constante d2 para n=2)
  - Banda dinamica 3-Sigma: T_t +/- 3 * sigma_imr
Tambem registra o desvio-padrao amostral global para comparacao metodologica.
"""
_garantir_gold(conn)
rows = [
    dict(r) for r in conn.execute("SELECT * FROM gold_receita_diaria").fetchall()
]
if not rows:
    return {
        "metrica": coluna_metrica,
        "media": 0.0,
        "desvio_padrao": 0.0,
        "desvio_padrao_amostral": 0.0,
        "mr_medio": 0.0,
        "lsc_3sigma": 0.0,
        "lic_3sigma": 0.0,
        "total_dias": 0,
        "dias_fora_controle": 0,
        "pontos_fora_controle": [],
        "serie": [],
    }

df = pd.DataFrame(rows)
serie_valores = df[coluna_metrica].astype(float)
media = float(serie_valores.mean())
desvio_amostral = float(serie_valores.std(ddof=1)) if len(df) > 1 else 0.0

# Ajuste sazonal por media movel centrada de 15 dias e estimador I-MR (d2 = 1.128)
tendencia = serie_valores.rolling(window=15, center=True, min_periods=1).mean()
residuo = serie_valores - tendencia
amplitude_movel = residuo.diff().abs()
mr_medio = float(amplitude_movel.mean()) if len(df) > 1 else 0.0
sigma_imr = (mr_medio / 1.128) if mr_medio > 0 else desvio_amostral

lsc_global = round(media + 3.0 * sigma_imr, 2)
lic_global = round(max(0.0, media - 3.0 * sigma_imr), 2)
media_round = round(media, 2)
sigma_imr_round = round(sigma_imr, 2)
desvio_amostral_round = round(desvio_amostral, 2)
mr_medio_round = round(mr_medio, 2)

df["media_processo"] = media_round
df["tendencia_sazonal"] = tendencia.round(2)
df["residuo_sazonal"] = residuo.round(2)
df["amplitude_movel_mr"] = amplitude_movel.fillna(0.0).round(2)
df["desvio_padrao"] = sigma_imr_round
df["desvio_padrao_amostral"] = desvio_amostral_round
df["lsc_3sigma"] = (tendencia + 3.0 * sigma_imr).round(2)
df["lic_3sigma"] = (tendencia - 3.0 * sigma_imr).clip(lower=0.0).round(2)
df["z_score"] = (
    (residuo / sigma_imr).round(2) if sigma_imr > 0 else 0.0
)
df["fora_de_controle"] = residuo.abs() > (3.0 * sigma_imr)
df["status_cep"] = df["fora_de_controle"].map(
    lambda x: "FORA_DE_CONTROLE" if x else "SOB_CONTROLE"
)

registros = df.to_dict(orient="records")
fora = [r for r in registros if r["fora_de_controle"]]

return {
    "metrica": coluna_metrica,
    "media": media_round,
    "desvio_padrao": sigma_imr_round,
    "desvio_padrao_amostral": desvio_amostral_round,
    "mr_medio": mr_medio_round,
    "lsc_3sigma": lsc_global,
    "lic_3sigma": lic_global,
    "total_dias": len(registros),
```

```
        "dias_fora_controle": len(fora),
        "pontos_fora_controle": fora,
        "serie": registros,
    }


def _salvar_csv(
    caminho: str, registros: list[dict[str, Any]], colunas: list[str] | None = None
) -> int:
    if not registros:
        return 0
    cols = colunas or list(registros[0].keys())
    with open(caminho, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=cols)
        writer.writeheader()
        writer.writerows(registros)
    return len(registros)


def exportar_camada_gold(
    conn: sqlite3.Connection, pasta_exports: str = PASTA_EXPORTS_PADRAO
) -> dict[str, int]:
    """Exporta todas as visoes Gold e a Carta de Controle para CSVs em exports/."""
    os.makedirs(pasta_exports, exist_ok=True)
    _garantir_gold(conn)

    consultas_views = [
        ("faturamento_categorias.csv", "SELECT * FROM gold_faturamento_categoria"),
        ("pareto_produtos_window.csv", "SELECT * FROM gold_pareto_produtos"),
        (
            "ltv_clientes.csv",
            "SELECT cliente, cidade, frequencia_pedidos AS total_pedidos, valor_monetario AS total_gasto FROM
gold_rfm_clientes",
        ),
        ("pedidos_consolidados.csv", "SELECT * FROM gold_resumo_pedidos ORDER BY data_pedido DESC"),
        ("receita_mensal_mom.csv", "SELECT * FROM gold_receita_mensal"),
        ("rfm_clientes.csv", "SELECT * FROM gold_rfm_clientes"),
        ("coorte_retencao.csv", "SELECT * FROM gold_coorte_retencao"),
        ("taxa_recompra.csv", "SELECT * FROM gold_taxa_recompra"),
        ("cancelamento_categoria_mes.csv", "SELECT * FROM gold_cancelamento_categoria_mes"),
        ("qualidade_rejeitados.csv", "SELECT * FROM gold_qualidade_resumo"),
    ]

    contagens: dict[str, int] = {}
    for nome_arquivo, sql in consultas_views:
        rows = [dict(r) for r in conn.execute(sql).fetchall()]
        caminho = os.path.join(pasta_exports, nome_arquivo)
        contagens[nome_arquivo] = _salvar_csv(caminho, rows)

    carta = calcular_carta_controle_diaria(conn)
    caminho_carta = os.path.join(pasta_exports, "carta_controle_diaria.csv")
    contagens["carta_controle_diaria.csv"] = _salvar_csv(
        caminho_carta, carta["serie"]
    )

    logger.info(
        "Exportacao Gold concluida em %s (%d arquivos gerados)",
        pasta_exports,
        len(contagens),
    )
    return contagens
```

Arquivo: `tests/test_queries.py`

```
"""
test_queries.py
---------------
Testes unitários automatizados com pytest.
Cria um banco SQLite em memória (:memory:), executa schema.sql e seed.sql,
```

```
e valida todas as consultas SQL do módulo queries.py.

Como rodar:
    pytest -v
"""

import os
import sqlite3
import sys

import pytest

# Adiciona src/ ao path para importar os módulos do projeto
_DIR_RAIZ = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(_DIR_RAIZ, "src"))

import queries

_DIR_SQL = os.path.join(_DIR_RAIZ, "sql")


@pytest.fixture
def conn():
    """Cria e popula um banco SQLite em memória isolado para cada teste."""
    conexao = sqlite3.connect(":memory:")
    conexao.row_factory = sqlite3.Row
    conexao.execute("PRAGMA foreign_keys = ON")

    for nome_arquivo in ("schema.sql", "seed.sql"):
        caminho = os.path.join(_DIR_SQL, nome_arquivo)
        with open(caminho, "r", encoding="utf-8") as f:
            conexao.executescript(f.read())

    yield conexao
    conexao.close()


def test_listar_produtos_retorna_todos(conn):
    """Deve retornar os 7 produtos cadastrados no seed."""
    produtos = queries.listar_produtos(conn)
    assert len(produtos) >= 5
    assert len(produtos) == 7
    assert "nome" in produtos[0].keys()  # noqa: SIM118
    assert "preco" in produtos[0].keys()  # noqa: SIM118


def test_filtrar_por_categoria_existente(conn):
    """Deve filtrar corretamente os produtos da categoria 'Curso' (case-insensitive)."""
    cursos = queries.filtrar_por_categoria(conn, "curso")
    assert len(cursos) == 4
    nomes = [c["nome"] for c in cursos]
    assert "Python do Zero ao Avançado" in nomes


def test_filtrar_por_categoria_inexistente(conn):
    """Deve retornar lista vazia para categoria que não existe."""
    resultado = queries.filtrar_por_categoria(conn, "Hardware")
    assert resultado == []


def test_listar_clientes_retorna_todos(conn):
    """Deve retornar os 7 clientes cadastrados no seed."""
    clientes = queries.listar_clientes(conn)
    assert len(clientes) >= 5
    assert len(clientes) == 7


def test_resumo_por_cliente_exclui_cancelados(conn):
    """Deve agregar compras por cliente ignorando pedidos cancelados."""
    resumo = queries.resumo_por_cliente(conn)
    assert len(resumo) == 6
```

```
    # Ana Lima tem 2 pedidos pagos somando R$ 575,00
    primeiro = resumo[0]
    assert primeiro["cliente"] == "Ana Lima"
    assert primeiro["total_pedidos"] == 2
    assert primeiro["total_gasto"] == 575.00


def test_produtos_mais_vendidos_ordenacao(conn):
    """Deve retornar o ranking ordenado pela quantidade de unidades vendidas."""
    ranking = queries.produtos_mais_vendidos(conn)
    assert len(ranking) > 0
    quantidades = [item["unidades_vendidas"] for item in ranking]
    assert quantidades == sorted(quantidades, reverse=True)
    assert ranking[0]["produto"] == "Template de Dashboard SQL"
    assert ranking[0]["unidades_vendidas"] == 5


def test_pedidos_detalhados_usa_view(conn):
    """Deve consultar a VIEW resumo_pedidos e retornar todos os 8 pedidos."""
    pedidos = queries.pedidos_detalhados(conn)
    assert len(pedidos) == 8
    assert "valor_total" in pedidos[0].keys()  # noqa: SIM118
    assert "total_itens" in pedidos[0].keys()  # noqa: SIM118


def test_faturamento_por_categoria(conn):
    """Deve agrupar receita e ticket médio pelas 3 categorias."""
    faturamento = queries.faturamento_por_categoria(conn)
    assert len(faturamento) == 3
    categorias = [f["categoria"] for f in faturamento]
    assert set(categorias) == {"Curso", "E-book", "Template"}


def test_clientes_acima_de_meta_having(conn):
    """Deve filtrar clientes com gasto acumulado acima da meta usando HAVING."""
    acima_200 = queries.clientes_acima_de_meta(conn, 200.0)
    assert len(acima_200) == 4
    for cliente in acima_200:
        assert cliente["total_gasto"] > 200.0


def test_produtos_acima_da_media_subquery(conn):
    """Deve retornar apenas produtos com preço superior à média geral."""
    acima_media = queries.produtos_acima_da_media(conn)
    assert len(acima_media) == 3
    for prod in acima_media:
        assert prod["diferenca_da_media"] > 0


def test_desempenho_produtos_cte_case_when(conn):
    """Deve classificar todos os produtos em Alto, Medio ou Baixo via CTE + CASE WHEN."""
    desempenho = queries.desempenho_produtos_cte(conn)
    assert len(desempenho) == 7
    rotulos = {item["desempenho"] for item in desempenho}
    assert rotulos.issubset({"Alto", "Medio", "Baixo"})
    assert desempenho[0]["produto"] == "Fundamentos de IA com Python"
    assert desempenho[0]["desempenho"] == "Alto"


def test_inserir_produto_cadastra_corretamente(conn):
    """Deve inserir um novo produto e aumentar o total de produtos cadastrados."""
    novo_id = queries.inserir_produto(conn, "Docker na Pratica", "Curso", 129.90, 50)
    assert novo_id == 8
    produtos = queries.listar_produtos(conn)
    assert len(produtos) == 8


def test_calcular_kpis_executivos(conn):
    """Deve calcular receita paga, ticket medio, LTV e taxas de conversao."""
    kpis = queries.calcular_kpis_executivos(conn)
    assert kpis["total_pedidos"] == 8
```

```
    assert kpis["receita_paga"] == 1141.00
    assert kpis["receita_pendente"] == 425.00
    assert kpis["taxa_conversao_pct"] == 62.5


def test_ranking_window_function_pareto(conn):
    """Deve calcular posicao no ranking, share % e acumulado Pareto com Window Functions."""
    ranking = queries.ranking_window_function(conn)
    assert len(ranking) == 7
    assert ranking[0]["posicao_ranking"] == 1
    assert ranking[0]["produto"] == "Fundamentos de IA com Python"
    assert ranking[-1]["pareto_acumulado_pct"] == 100.0
```

Arquivo: `tests/test_qualidade.py`

```
"""
Testes unitarios para o motor de validacao de qualidade de dados (qualidade.py).
Cobre individualmente todas as regras da camada Silver e o relatorio de quarentena.
"""

import os
import sqlite3
import sys

_DIR_RAIZ = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(_DIR_RAIZ, "src"))

from pipeline.qualidade import (
    gerar_relatorio_qualidade,
    validar_chave_duplicada,
    validar_data_iso,
    validar_email,
    validar_fk_existente,
    validar_inteiro_nao_negativo,
    validar_numero_positivo,
    validar_obrigatorios,
    validar_status_pedido,
)


def test_validar_obrigatorios_sucesso_e_falha():
    reg_ok = {"id": "1", "nome": "Ana", "email": "ana@teste.com"}
    ok, motivo = validar_obrigatorios(reg_ok, ["id", "nome", "email"])
    assert ok is True
    assert motivo is None

    reg_nulo = {"id": "1", "nome": "   ", "email": "ana@teste.com"}
    ok2, motivo2 = validar_obrigatorios(reg_nulo, ["id", "nome", "email"])
    assert ok2 is False
    assert motivo2 == "nulo_obrigatorio:nome"


def test_validar_chave_duplicada_e_invalida():
    vistos = {1, 2, 3}
    ok, motivo = validar_chave_duplicada("4", vistos)
    assert ok is True
    assert motivo is None

    ok_dup, motivo_dup = validar_chave_duplicada("2", vistos)
    assert ok_dup is False
    assert motivo_dup == "chave_duplicada"

    ok_inv, motivo_inv = validar_chave_duplicada("abc", vistos)
    assert ok_inv is False
    assert motivo_inv == "chave_invalida"

    ok_neg, motivo_neg = validar_chave_duplicada("-5", vistos)
```

```
    assert ok_neg is False
    assert motivo_neg == "chave_invalida"

    ok_float, motivo_float = validar_chave_duplicada("4.0", vistos)
    assert ok_float is False
    assert motivo_float == "chave_invalida"


def test_validar_numero_positivo():
    assert validar_numero_positivo("197.50", "preco") == (True, None)
    assert validar_numero_positivo("0", "preco") == (
        False,
        "valor_menor_ou_igual_zero:preco",
    )
    assert validar_numero_positivo("-10", "quantidade") == (
        False,
        "valor_menor_ou_igual_zero:quantidade",
    )
    assert validar_numero_positivo("texto", "preco") == (
        False,
        "valor_nao_numerico:preco",
    )
    assert validar_numero_positivo("nan", "preco") == (
        False,
        "valor_nao_numerico:preco",
    )
    assert validar_numero_positivo("inf", "preco") == (
        False,
        "valor_nao_numerico:preco",
    )


def test_validar_inteiro_nao_negativo():
    assert validar_inteiro_nao_negativo("0", "estoque") == (True, None)
    assert validar_inteiro_nao_negativo("100", "estoque") == (True, None)
    assert validar_inteiro_nao_negativo("-1", "estoque") == (
        False,
        "inteiro_negativo:estoque",
    )
    assert validar_inteiro_nao_negativo("12.5", "estoque") == (
        False,
        "inteiro_invalido:estoque",
    )


def test_validar_status_pedido():
    for st in ("pago", "pendente", "cancelado", "PAGO"):
        assert validar_status_pedido(st) == (True, None)

    assert validar_status_pedido("em_analise") == (False, "status_fora_do_dominio")
    assert validar_status_pedido("") == (False, "status_fora_do_dominio")


def test_validar_email_formato_e_duplicidade():
    emails = {"ana@exemplo.com.br"}
    assert validar_email("novo.cliente@dominio.com", emails) == (True, None)
    assert validar_email("ana@exemplo.com.br", emails) == (False, "email_duplicado")
    assert validar_email("email_sem_arroba.com", emails) == (
        False,
        "email_formato_invalido",
    )


def test_validar_data_iso():
    assert validar_data_iso("2026-10-20", "data_pedido") == (True, None)
    assert validar_data_iso("2026-02-30", "data_pedido") == (
        False,
        "data_invalida:data_pedido",
    )
    assert validar_data_iso("20/10/2026", "criado_em") == (
        False,
```

```
        "data_invalida:criado_em",
    )
    assert validar_data_iso("1900-01-01", "data_pedido") == (
        False,
        "data_fora_janela_negocio:data_pedido",
    )
    assert validar_data_iso("2099-12-31", "criado_em") == (
        False,
        "data_fora_janela_negocio:criado_em",
    )


def test_validar_fk_existente():
    pais = {10, 20, 30}
    assert validar_fk_existente("20", pais, "cliente_id") == (True, None)
    assert validar_fk_existente("99", pais, "cliente_id") == (
        False,
        "fk_orfa:cliente_id",
    )
    assert validar_fk_existente("invalido", pais, "produto_id") == (
        False,
        "fk_orfa:produto_id",
    )


def test_gerar_relatorio_qualidade_exporta_csv(tmp_path):
    conn = sqlite3.connect(":memory:")
    conn.row_factory = sqlite3.Row
    conn.execute(
        """
        CREATE TABLE silver_rejeitados (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tabela_origem TEXT NOT NULL,
            motivo TEXT NOT NULL,
            registro_raw TEXT NOT NULL
        )
        """
    )
    conn.executemany(
        "INSERT INTO silver_rejeitados (tabela_origem, motivo, registro_raw) VALUES (?, ?, ?)",
        [
            ("bronze_pedidos", "status_fora_do_dominio", "{}"),
            ("bronze_pedidos", "status_fora_do_dominio", "{}"),
            ("bronze_clientes", "email_formato_invalido", "{}"),
        ],
    )
    resumo = gerar_relatorio_qualidade(conn, pasta_exports=str(tmp_path))
    assert len(resumo) == 2
    assert resumo[0]["motivo"] == "status_fora_do_dominio"
    assert resumo[0]["total_rejeitados"] == 2
    assert (tmp_path / "qualidade_rejeitados.csv").exists()
    conn.close()
```

Arquivo: `tests/test_pipeline.py`

```
"""
Testes automatizados para o pipeline da Arquitetura Medalha (Bronze -> Silver -> Gold),
determinismo do Faker, idempotencia, KPIs Gold, carta de controle 3-Sigma e CLI/Menu.
"""

import os
import sqlite3
import sys
from pathlib import Path
from unittest.mock import patch

import pytest

_DIR_RAIZ = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
sys.path.insert(0, os.path.join(_DIR_RAIZ, "src"))

import database
import main
import queries
import relatorios
from pipeline.executar import run_pipeline
from pipeline.gerar_dados import gerar_arquivos_raw
from pipeline.gold import calcular_carta_controle_diaria


def test_gerador_dados_deterministico_mesma_semente(tmp_path: Path) -> None:
    """Garante que a mesma semente gera exatamente os mesmos arquivos CSV."""
    pasta1 = tmp_path / "raw1"
    pasta2 = tmp_path / "raw2"

    c1 = gerar_arquivos_raw(str(pasta1), semente=42, num_clientes=30, num_pedidos=100)
    c2 = gerar_arquivos_raw(str(pasta2), semente=42, num_clientes=30, num_pedidos=100)

    assert c1 == c2
    for nome_arq in ["clientes.csv", "produtos.csv", "pedidos.csv", "itens_pedido.csv"]:
        conteudo1 = (pasta1 / nome_arq).read_text(encoding="utf-8")
        conteudo2 = (pasta2 / nome_arq).read_text(encoding="utf-8")
        assert conteudo1 == conteudo2


@pytest.fixture(scope="module")
def pipeline_isolado(tmp_path_factory: pytest.TempPathFactory):
    """Executa o pipeline medalha completo em banco temporario para testes rapidos."""
    base_dir = tmp_path_factory.mktemp("medalha_suite")
    caminho_banco = base_dir / "medalha_test.db"
    pasta_raw = base_dir / "raw"
    pasta_exports = base_dir / "exports"

    banco_original_db = database.CAMINHO_BANCO
    banco_original_main = main.CAMINHO_BANCO
    database.CAMINHO_BANCO = str(caminho_banco)
    main.CAMINHO_BANCO = str(caminho_banco)

    database.criar_banco()
    conn = database.conectar()

    resumo = run_pipeline(
        conn=conn,
        pasta_raw=str(pasta_raw),
        pasta_exports=str(pasta_exports),
        semente=42,
        regerar_csv=True,
    )
    yield {
        "conn": conn,
        "caminho_banco": str(caminho_banco),
        "pasta_raw": str(pasta_raw),
        "pasta_exports": str(pasta_exports),
        "resumo": resumo,
    }
    conn.close()
    database.CAMINHO_BANCO = banco_original_db
    main.CAMINHO_BANCO = banco_original_main


def test_bronze_silver_gold_fluxo_completo_e_quarentena(pipeline_isolado: dict) -> None:
    """Verifica contagens de Bronze, Silver, quarentena por motivo e views Gold."""
    conn: sqlite3.Connection = pipeline_isolado["conn"]
    resumo = pipeline_isolado["resumo"]

    assert resumo["bronze"]["bronze_clientes"] == 205
    assert resumo["bronze"]["bronze_produtos"] == 34
    assert resumo["bronze"]["bronze_pedidos"] == 5075

    assert resumo["silver"]["clientes"]["validas"] == 200
```

```
    assert resumo["silver"]["clientes"]["rejeitadas"] == 5
    assert resumo["silver"]["produtos"]["validas"] == 30
    assert resumo["silver"]["produtos"]["rejeitadas"] == 4
    assert resumo["silver"]["pedidos"]["validas"] == 5000
    assert resumo["silver"]["pedidos"]["rejeitadas"] == 75
    assert resumo["silver"]["itens_pedido"]["rejeitadas"] == 60

    # Verifica motivos registrados em silver_rejeitados
    relatorio_qual = queries.resumo_qualidade_dados(conn)
    assert len(relatorio_qual) > 0
    motivos_encontrados = {r["motivo"] for r in relatorio_qual}
    assert "chave_duplicada" in motivos_encontrados
    assert "email_formato_invalido" in motivos_encontrados
    assert "status_fora_do_dominio" in motivos_encontrados
    assert "fk_orfa:cliente_id" in motivos_encontrados

    # Verifica arquivo de auditoria de qualidade exportado
    arq_qualidade = Path(pipeline_isolado["pasta_exports"]) / "qualidade_rejeitados.csv"
    assert arq_qualidade.exists()


def test_idempotencia_pipeline_duas_execucoes(pipeline_isolado: dict) -> None:
    """Rodar o pipeline duas vezes deve manter exatamente as mesmas contagens e registrar auditoria."""
    conn: sqlite3.Connection = pipeline_isolado["conn"]
    pasta_raw = pipeline_isolado["pasta_raw"]
    pasta_exports = pipeline_isolado["pasta_exports"]

    resumo_1 = pipeline_isolado["resumo"]
    resumo_2 = run_pipeline(
        conn=conn,
        pasta_raw=pasta_raw,
        pasta_exports=pasta_exports,
        semente=42,
        regerar_csv=False,
    )

    assert resumo_1["bronze"] == resumo_2["bronze"]
    assert resumo_1["silver"] == resumo_2["silver"]
    assert len(resumo_1["qualidade"]) == len(resumo_2["qualidade"])
    assert resumo_1["execucao"]["modo_geracao"] == "regerado"
    assert resumo_2["execucao"]["modo_geracao"] == "reprocessado"

    historico = queries.historico_execucoes_pipeline(conn)
    assert len(historico) >= 2
    assert historico[0]["status"] == "sucesso"
    assert historico[0]["registros_rejeitados"] == 144


def test_kpis_gold_mom_rfm_coorte_cancelamento_e_carta_controle(pipeline_isolado: dict) -> None:
    """Valida todas as novas consultas Gold e a carta de controle 3-Sigma."""
    conn: sqlite3.Connection = pipeline_isolado["conn"]

    mom = queries.receita_mensal_mom(conn)
    assert len(mom) >= 10
    assert "variacao_mom_pct" in dict(mom[0])
    assert "media_movel_3m" in dict(mom[0])

    rfm = queries.segmentacao_rfm(conn)
    assert len(rfm) > 0
    segmentos = {r["segmento_rfm"] for r in rfm}
    assert segmentos.issubset({"Campeoes", "Leais", "Em risco", "Inativos"})

    resumo_rfm = queries.resumo_segmentos_rfm(conn)
    assert sum(r["total_clientes"] for r in resumo_rfm) == len(rfm)

    coortes = queries.retencao_coorte(conn)
    assert len(coortes) >= 6
    assert all(0 <= c["taxa_retencao_pct"] <= 100 for c in coortes)

    recompra = queries.taxa_recompra_global(conn)
    assert 0 < recompra["taxa_recompra_pct"] < 100
```

```
    assert recompra["clientes_recorrentes"] > 0

    # Reconciliacao aritmetica exata da base paga (status = 'pago')
    kpis = queries.calcular_kpis_executivos(conn)
    assert round(kpis["receita_paga"] / kpis["pedidos_pagos"], 2) == kpis["ticket_medio_pedido"]
    assert round(kpis["receita_paga"] / recompra["clientes_compradores"], 2) == kpis["ltv_medio_cliente"]
    assert round(sum(r["receita_segmento"] for r in resumo_rfm), 2) == kpis["receita_paga"]
    assert round(sum(r["receita_liquida"] for r in mom), 2) == kpis["receita_paga"]

    canc_cat = queries.cancelamento_categoria_mes(conn)
    assert len(canc_cat) > 0
    assert "taxa_cancelamento_pct" in dict(canc_cat[0])

    carta = calcular_carta_controle_diaria(conn)
    assert carta["total_dias"] > 100
    assert carta["lsc_3sigma"] > carta["media"]
    assert carta["desvio_padrao"] < carta["desvio_padrao_amostral"]
    assert carta["dias_fora_controle"] == 2


def test_exportacoes_e_dashboard_executivo(pipeline_isolado: dict) -> None:
    """Testa exportacoes CSV e geracao do dashboard executivo em PNG."""
    conn: sqlite3.Connection = pipeline_isolado["conn"]
    relatorios.exportar_relatorios_csv(conn)
    relatorios.gerar_grafico_faturamento(conn)
    caminho_img = Path(__file__).resolve().parent.parent / "assets" / "dashboard_executivo.png"
    assert caminho_img.exists()
    assert caminho_img.stat().st_size > 10000


def test_menu_opcoes_1_a_20_e_entradas_invalidas(pipeline_isolado: dict) -> None:
    """Garante que o menu interativo navega pelas opcoes 1 a 20 e trata entradas invalidas."""
    entradas = iter([
        "1",
        "2", "1",
        "3",
        "4",
        "5",
        "6",
        "7",
        "8", "500",
        "9",
        "10",
        "11",
        "12", "Curso Menu Teste", "1", "199.90", "50",
        "14",
        "16",
        "17",
        "18",
        "19",
        "20",
        "99",
        "0",
    ])
    with patch("builtins.input", lambda _: next(entradas)):
        main.main([])


def test_cli_parse_args_e_clone_limpo_sem_banco(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> None:
    """Testa parsing de flags e execucao do modo 2 (sem --pipeline e sem .db previo)."""
    args_pipe = main._parse_args(["--pipeline"])
    assert args_pipe.pipeline is True
    assert args_pipe.reprocessar is False
    assert args_pipe.demo is False

    args_reproc = main._parse_args(["--reprocessar"])
    assert args_reproc.reprocessar is True
    assert args_reproc.pipeline is False

    args_demo = main._parse_args(["--demo"])
    assert args_demo.demo is True
```

```
    assert args_demo.pipeline is False

    # Testa modo 2 em clone limpo (quando o arquivo .db nao existe ainda)
    banco_inexistente = tmp_path / "clone_limpo_sem_db.db"
    assert not banco_inexistente.exists()
    monkeypatch.setattr(database, "CAMINHO_BANCO", str(banco_inexistente))
    monkeypatch.setattr(main, "CAMINHO_BANCO", str(banco_inexistente))
    entradas_modo2 = iter(["1", "11", "16", "17", "18", "19", "20", "0"])
    with patch("builtins.input", lambda _: next(entradas_modo2)):
        main.main([])
    assert banco_inexistente.exists()
```